

# Reviewer-Revised Research Report: Transformation Vectors and Composition Diagnostics

## Date

2026-06-12

## Purpose

Fix the reviewer-revised research state for external verification. This version incorporates AC-level review concerns: endpoint leakage, syntax-holdout overclaiming, McNemar tests, semantic-control statistics, and the terminology change to a third-order signed permutation coherence test.

## Track 1 fixed result

Delta vectors add reproducible information beyond target-only endpoints in the multiseed full-semantic setting. The syntax=1.0 result has now been directly ablated: y\_only also reaches 1.0, so the syntax split is interpreted as endpoint/surface leakage rather than as geometry. BERT-large and DeBERTa-v3-base spot-checks both rank delta best under Linear SVC and logistic regression. UPAT is now reported as a hard-holdout boundary where delta is not reliably better than y\_only.

## Track 2 fixed result

Composition order matters. Semantic equivalence shifts noncommutativity distributions with strong statistical tests. The third-order result is now framed as signed permutation coherence, not a formal Jacobi identity. Decoder pairwise composition is now included. After multiple-testing correction, QMT is the only below-null triple that is stable across all five tested models.

## Main conclusion

The evidence supports a local QMT signed-permutation cancellation effect across tested encoder models, while negation-heavy triples expose the boundary of the phenomenon. The current work is promising but not submission-ready without the listed blocker experiments.

# Delta vs endpoint ablation with McNemar evidence

| model              | delta_mean | delta_std | y_only_mean | y_only_std | concat_mean | concat_std | x_only_mean | diff_delta_y | diff_delta_concat | max_mcnemar_p | max_mcnemar_sig_0_001 |
|--------------------|------------|-----------|-------------|------------|-------------|------------|-------------|--------------|-------------------|---------------|-----------------------|
| bert-base-uncased  | 0.9075     | 0.0117    | 0.8632      | 0.0167     | 0.8969      | 0.0185     | 0.1667      | 0.0443       | 0.0106            | 0.0000        | 1.0000                |
| distilgpt2         | 0.8572     | 0.0125    | 0.7265      | 0.0094     | 0.7604      | 0.0107     | 0.1667      | 0.1307       | 0.0968            | 0.0000        | 1.0000                |
| distilroberta-base | 0.8810     | 0.0053    | 0.8306      | 0.0056     | 0.8417      | 0.0060     | 0.1667      | 0.0504       | 0.0394            | 0.0000        | 1.0000                |
| gpt2               | 0.8332     | 0.0076    | 0.7496      | 0.0152     | 0.7765      | 0.0093     | 0.1667      | 0.0836       | 0.0567            | 0.0000        | 1.0000                |
| roberta-base       | 0.8817     | 0.0090    | 0.8391      | 0.0072     | 0.8666      | 0.0089     | 0.1667      | 0.0426       | 0.0151            | 0.0000        | 1.0000                |

Syntax representation ablation: y\_only solves the split

| model              | classifier | concat | delta  | x_only | y_only |
|--------------------|------------|--------|--------|--------|--------|
| bert-base-uncased  | linear_svc | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| bert-base-uncased  | logreg     | 0.9904 | 0.9910 | 0.1667 | 0.9861 |
| distilgpt2         | linear_svc | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| distilgpt2         | logreg     | 0.9976 | 1.0000 | 0.1667 | 0.9957 |
| distilroberta-base | linear_svc | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| distilroberta-base | logreg     | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| gpt2               | linear_svc | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| gpt2               | logreg     | 0.9979 | 0.9924 | 0.1667 | 0.9876 |
| roberta-base       | linear_svc | 1.0000 | 1.0000 | 0.1667 | 1.0000 |
| roberta-base       | logreg     | 1.0000 | 1.0000 | 0.1667 | 1.0000 |

Layerwise/pooling syntax sanity check: top rows

| classifier | accuracy | model             | layer   | pooling    | representation | n_base   | n_train   | n_test    |
|------------|----------|-------------------|---------|------------|----------------|----------|-----------|-----------|
| linear_svc | 1.0000   | bert-base-uncased | 0.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 0.0000  | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 2.0000  | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 1.0000  | cls        | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 1.0000  | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 1.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 2.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 4.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 4.0000  | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 3.0000  | cls        | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 3.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 3.0000  | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 2.0000  | cls        | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 2.0000  | cls        | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 12.0000 | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 7.0000  | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 6.0000  | last_token | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 6.0000  | cls        | delta          | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 11.0000 | mean       | y_only         | 200.0000 | 2400.0000 | 4800.0000 |
| linear_svc | 1.0000   | bert-base-uncased | 11.0000 | mean       | delta          | 200.0000 | 2400.0000 | 4800.0000 |

# DeBERTa-v3-small modern spot-check

| model                      | classifier | concat | delta  | x_only | y_only |
|----------------------------|------------|--------|--------|--------|--------|
| microsoft/deberta-v3-small | linear_svc | 0.8225 | 0.8708 | 0.1667 | 0.8042 |
| microsoft/deberta-v3-small | logreg     | 0.8283 | 0.7958 | 0.1667 | 0.7700 |

# BERT-large and DeBERTa-v3-base spot-checks

| model                     | classifier | concat | delta  | x_only | y_only |
|---------------------------|------------|--------|--------|--------|--------|
| bert-large-uncased        | linear_svc | 0.8508 | 0.9025 | 0.1667 | 0.8375 |
| bert-large-uncased        | logreg     | 0.7600 | 0.8542 | 0.1667 | 0.7500 |
| microsoft/deberta-v3-base | linear_svc | 0.7758 | 0.8117 | 0.1667 | 0.7467 |
| microsoft/deberta-v3-base | logreg     | 0.6942 | 0.7517 | 0.1667 | 0.7258 |

# UPAT hard-holdout representation ablation

| model              | feature   | acc    | f1     | baseline |
|--------------------|-----------|--------|--------|----------|
| bert-base-uncased  | x_only    | 0.2000 | 0.0667 | 0.2000   |
| bert-base-uncased  | y_only    | 0.8250 | 0.8266 | 0.2000   |
| bert-base-uncased  | delta     | 0.7250 | 0.7253 | 0.2000   |
| bert-base-uncased  | concat_xy | 0.8000 | 0.7974 | 0.2000   |
| distilroberta-base | x_only    | 0.2000 | 0.0667 | 0.2000   |
| distilroberta-base | y_only    | 0.6500 | 0.6105 | 0.2000   |
| distilroberta-base | delta     | 0.7250 | 0.6530 | 0.2000   |
| distilroberta-base | concat_xy | 0.6250 | 0.5703 | 0.2000   |
| roberta-base       | x_only    | 0.2000 | 0.0667 | 0.2000   |
| roberta-base       | y_only    | 0.7250 | 0.7210 | 0.2000   |
| roberta-base       | delta     | 0.6750 | 0.6333 | 0.2000   |
| roberta-base       | concat_xy | 0.7250 | 0.7185 | 0.2000   |
| gpt2               | x_only    | 0.2000 | 0.0667 | 0.2000   |
| gpt2               | y_only    | 0.4750 | 0.4358 | 0.2000   |
| gpt2               | delta     | 0.4250 | 0.3481 | 0.2000   |
| gpt2               | concat_xy | 0.4750 | 0.4338 | 0.2000   |
| distilgpt2         | x_only    | 0.2000 | 0.0959 | 0.2000   |
| distilgpt2         | y_only    | 0.6000 | 0.5441 | 0.2000   |
| distilgpt2         | delta     | 0.6500 | 0.5953 | 0.2000   |
| distilgpt2         | concat_xy | 0.5750 | 0.5169 | 0.2000   |

# UPAT delta vs y\_only McNemar tests

| model              | compare         | delta_acc | y_only_acc | diff    | b_delta_correct_y_wrong | c_delta_wrong_y_correct | p_value |
|--------------------|-----------------|-----------|------------|---------|-------------------------|-------------------------|---------|
| bert-base-uncased  | delta_vs_y_only | 0.7250    | 0.8250     | -0.1000 | 2.0000                  | 6.0000                  | 0.2891  |
| distilroberta-base | delta_vs_y_only | 0.7250    | 0.6500     | 0.0750  | 4.0000                  | 1.0000                  | 0.3750  |
| roberta-base       | delta_vs_y_only | 0.6750    | 0.7250     | -0.0500 | 7.0000                  | 9.0000                  | 0.8036  |
| gpt2               | delta_vs_y_only | 0.4250    | 0.4750     | -0.0500 | 7.0000                  | 9.0000                  | 0.8036  |
| distilgpt2         | delta_vs_y_only | 0.6500    | 0.6000     | 0.0500  | 5.0000                  | 3.0000                  | 0.7266  |

# Full-semantic pooling ablation

| model             | pooling    | classifier | concat | delta  | x_only | y_only |
|-------------------|------------|------------|--------|--------|--------|--------|
| bert-base-uncased | cls        | linear_svc | 0.8269 | 0.8775 | 0.1667 | 0.7397 |
| bert-base-uncased | cls        | logreg     | 0.7031 | 0.7200 | 0.1667 | 0.6781 |
| bert-base-uncased | last_token | linear_svc | 0.7564 | 0.8514 | 0.1667 | 0.7508 |
| bert-base-uncased | last_token | logreg     | 0.6792 | 0.7103 | 0.1667 | 0.6847 |
| bert-base-uncased | mean       | linear_svc | 0.8850 | 0.8958 | 0.1667 | 0.8806 |
| bert-base-uncased | mean       | logreg     | 0.8725 | 0.8844 | 0.1667 | 0.8603 |
| gpt2              | cls        | linear_svc | 0.3683 | 0.3867 | 0.1667 | 0.3683 |
| gpt2              | cls        | logreg     | 0.3422 | 0.3203 | 0.1667 | 0.3683 |
| gpt2              | last_token | linear_svc | 0.7469 | 0.7564 | 0.1667 | 0.7200 |
| gpt2              | last_token | logreg     | 0.6453 | 0.6683 | 0.1667 | 0.6253 |
| gpt2              | mean       | linear_svc | 0.7822 | 0.8450 | 0.1667 | 0.7539 |
| gpt2              | mean       | logreg     | 0.7803 | 0.7492 | 0.1667 | 0.7633 |
| roberta-base      | cls        | linear_svc | 0.8236 | 0.8736 | 0.1667 | 0.7911 |
| roberta-base      | cls        | logreg     | 0.7944 | 0.8514 | 0.1667 | 0.7669 |
| roberta-base      | last_token | linear_svc | 0.7656 | 0.7650 | 0.1667 | 0.7042 |
| roberta-base      | last_token | logreg     | 0.7056 | 0.6756 | 0.1667 | 0.6739 |
| roberta-base      | mean       | linear_svc | 0.8631 | 0.8786 | 0.1667 | 0.8431 |
| roberta-base      | mean       | logreg     | 0.8150 | 0.8092 | 0.1667 | 0.8017 |

# Confusion analysis: negation vs non-negation recall

| source        | model              | classifier | group        | mean_recall | mean_error_rate | n_classes |
|---------------|--------------------|------------|--------------|-------------|-----------------|-----------|
| full_semantic | bert-base-uncased  | linear_svc | negation     | 1.0000      | 0.0000          | 1.0000    |
| full_semantic | bert-base-uncased  | linear_svc | non_negation | 0.8725      | 0.1275          | 5.0000    |
| full_semantic | bert-base-uncased  | logreg     | negation     | 0.9862      | 0.0138          | 1.0000    |
| full_semantic | bert-base-uncased  | logreg     | non_negation | 0.8817      | 0.1182          | 5.0000    |
| full_semantic | distilgpt2         | linear_svc | negation     | 1.0000      | 0.0000          | 1.0000    |
| full_semantic | distilgpt2         | linear_svc | non_negation | 0.8020      | 0.1980          | 5.0000    |
| full_semantic | distilgpt2         | logreg     | negation     | 0.9725      | 0.0275          | 1.0000    |
| full_semantic | distilgpt2         | logreg     | non_negation | 0.7295      | 0.2705          | 5.0000    |
| full_semantic | distilroberta-base | linear_svc | negation     | 1.0000      | 0.0000          | 1.0000    |
| full_semantic | distilroberta-base | linear_svc | non_negation | 0.8470      | 0.1530          | 5.0000    |
| full_semantic | distilroberta-base | logreg     | negation     | 1.0000      | 0.0000          | 1.0000    |
| full_semantic | distilroberta-base | logreg     | non_negation | 0.8252      | 0.1747          | 5.0000    |
| full_semantic | gpt2               | linear_svc | negation     | 0.9600      | 0.0400          | 1.0000    |
| full_semantic | gpt2               | linear_svc | non_negation | 0.7973      | 0.2027          | 5.0000    |
| full_semantic | gpt2               | logreg     | negation     | 0.7312      | 0.2687          | 1.0000    |
| full_semantic | gpt2               | logreg     | non_negation | 0.7410      | 0.2590          | 5.0000    |
| full_semantic | roberta-base       | linear_svc | negation     | 0.9988      | 0.0012          | 1.0000    |
| full_semantic | roberta-base       | linear_svc | non_negation | 0.8570      | 0.1430          | 5.0000    |
| full_semantic | roberta-base       | logreg     | negation     | 0.9300      | 0.0700          | 1.0000    |
| full_semantic | roberta-base       | logreg     | non_negation | 0.7760      | 0.2240          | 5.0000    |

# Third-order signed permutation summary with bootstrap CI

| model              | triple | mean_relative_jacobi_norm | relative_jacobi_norm_ci_95_low | relative_jacobi_norm_ci_95_high | mean_jacobi_to_null_mean_ratio | jacobi_to_null_mean_ratio_ci_95_low | jacobi_to_null_mean_ratio_ci_95_high | mean_percentile_smaller_is | n        |
|--------------------|--------|---------------------------|--------------------------------|---------------------------------|--------------------------------|-------------------------------------|--------------------------------------|----------------------------|----------|
| bert-base-uncased  | NMT    | 0.3048                    | 0.3005                         | 0.3090                          | 0.7750                         | 0.7669                              | 0.7829                               | 0.3522                     | 100.0000 |
| bert-base-uncased  | NQM    | 0.2695                    | 0.2660                         | 0.2727                          | 0.8628                         | 0.8533                              | 0.8720                               | 0.3850                     | 100.0000 |
| bert-base-uncased  | NQT    | 0.3719                    | 0.3674                         | 0.3763                          | 1.1170                         | 1.1082                              | 1.1259                               | 0.7005                     | 100.0000 |
| bert-base-uncased  | QMT    | 0.2760                    | 0.2712                         | 0.2807                          | 0.6833                         | 0.6773                              | 0.6892                               | 0.1018                     | 100.0000 |
| distilroberta-base | NMT    | 0.3522                    | 0.3485                         | 0.3558                          | 1.1338                         | 1.1276                              | 1.1404                               | 0.7066                     | 100.0000 |
| distilroberta-base | NQM    | 0.2847                    | 0.2809                         | 0.2882                          | 1.0821                         | 1.0735                              | 1.0914                               | 0.7777                     | 100.0000 |
| distilroberta-base | NQT    | 0.2777                    | 0.2752                         | 0.2802                          | 1.0803                         | 1.0765                              | 1.0841                               | 0.6274                     | 100.0000 |
| distilroberta-base | QMT    | 0.1849                    | 0.1832                         | 0.1866                          | 0.6311                         | 0.6267                              | 0.6356                               | 0.0997                     | 100.0000 |
| roberta-base       | NMT    | 0.4315                    | 0.4255                         | 0.4371                          | 1.2532                         | 1.2423                              | 1.2637                               | 0.8057                     | 100.0000 |
| roberta-base       | NQM    | 0.3122                    | 0.3086                         | 0.3158                          | 0.9799                         | 0.9743                              | 0.9853                               | 0.6475                     | 100.0000 |
| roberta-base       | NQT    | 0.3452                    | 0.3413                         | 0.3490                          | 1.0805                         | 1.0739                              | 1.0871                               | 0.7059                     | 100.0000 |
| roberta-base       | QMT    | 0.2190                    | 0.2160                         | 0.2222                          | 0.6229                         | 0.6168                              | 0.6287                               | 0.1013                     | 100.0000 |

Decoder signed permutation replication

| model      | triple | mean_jacobi_norm | mean_relative_jacobi_norm | null_relative_jacobi_norm | jacobi_to_null_mean_ratio | mean_percentile_smaller | n        | relative_jacobi_norm | relative_jacobi_norm | jacobi_to_null_mean_ratio | jacobi_to_null_mean_ratio_c95 |
|------------|--------|------------------|---------------------------|---------------------------|---------------------------|-------------------------|----------|----------------------|----------------------|---------------------------|-------------------------------|
| distilgpt2 | NMT    | 10.3687          | 0.2276                    | 0.2387                    | 0.9507                    | 0.5781                  | 100.0000 | 0.2171               | 0.2388               | 0.9284                    | 0.9722                        |
| distilgpt2 | NQM    | 9.5684           | 0.2144                    | 0.2540                    | 0.8464                    | 0.4061                  | 100.0000 | 0.2056               | 0.2239               | 0.8249                    | 0.8692                        |
| distilgpt2 | NQT    | 15.8990          | 0.3385                    | 0.2403                    | 1.4202                    | 0.8621                  | 100.0000 | 0.3241               | 0.3537               | 1.3820                    | 1.4555                        |
| distilgpt2 | QMT    | 9.8263           | 0.2492                    | 0.3179                    | 0.7708                    | 0.3207                  | 100.0000 | 0.2320               | 0.2668               | 0.7446                    | 0.7972                        |
| gpt2       | NMT    | 31.1167          | 0.2894                    | 0.2137                    | 1.3348                    | 0.7670                  | 100.0000 | 0.2736               | 0.3065               | 1.2981                    | 1.3723                        |
| gpt2       | NQM    | 24.9467          | 0.2799                    | 0.2315                    | 1.1947                    | 0.7302                  | 100.0000 | 0.2642               | 0.2955               | 1.1675                    | 1.2221                        |
| gpt2       | NQT    | 16.4593          | 0.1562                    | 0.1777                    | 0.8666                    | 0.4331                  | 100.0000 | 0.1466               | 0.1658               | 0.8356                    | 0.9004                        |
| gpt2       | QMT    | 14.3721          | 0.1705                    | 0.3167                    | 0.5393                    | 0.1852                  | 100.0000 | 0.1621               | 0.1797               | 0.5192                    | 0.5611                        |

Signed permutation multiple-testing correction

| model              | triple | n        | mean_ratio_to_nullboot | bootstrap_p_ratio_less_than | direction  | bonferroni_p | bh_fdr_p | passes_bonferroni_0_05 | passes_bh_fdr_0_05 |
|--------------------|--------|----------|------------------------|-----------------------------|------------|--------------|----------|------------------------|--------------------|
| bert-base-uncased  | NMT    | 100.0000 | 0.7750                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| bert-base-uncased  | NQM    | 100.0000 | 0.8628                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| bert-base-uncased  | QMT    | 100.0000 | 0.6833                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| distilgpt2         | NQM    | 100.0000 | 0.8464                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| distilgpt2         | QMT    | 100.0000 | 0.7708                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| distilroberta-base | QMT    | 100.0000 | 0.6311                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| gpt2               | NQT    | 100.0000 | 0.8666                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| gpt2               | QMT    | 100.0000 | 0.5393                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| roberta-base       | NQM    | 100.0000 | 0.9799                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| roberta-base       | QMT    | 100.0000 | 0.6229                 | 0.0000                      | below_null | 0.0010       | 0.0001   | 1.0000                 | 1.0000             |
| distilgpt2         | NMT    | 100.0000 | 0.9507                 | 0.0001                      | below_null | 0.0020       | 0.0002   | 1.0000                 | 1.0000             |
| bert-base-uncased  | NQT    | 100.0000 | 1.1170                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| distilgpt2         | NQT    | 100.0000 | 1.4202                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| distilroberta-base | NMT    | 100.0000 | 1.1338                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| distilroberta-base | NQM    | 100.0000 | 1.0821                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| distilroberta-base | NQT    | 100.0000 | 1.0803                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| gpt2               | NMT    | 100.0000 | 1.3348                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| gpt2               | NQM    | 100.0000 | 1.1947                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| roberta-base       | NMT    | 100.0000 | 1.2532                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |
| roberta-base       | NQT    | 100.0000 | 1.0805                 | 1.0000                      | above_null | 1.0000       | 1.0000   | 0.0000                 | 0.0000             |

# Decoder pairwise composition summary

| model      | pair     | mean_commutator_norm | std_commutator_norm | mean_relative_commutator_norm | std_relative_commutator_norm | mean_cosine | std_cosine | mean_noncommutativity | std_noncommutativity | n       |
|------------|----------|----------------------|---------------------|-------------------------------|------------------------------|-------------|------------|-----------------------|----------------------|---------|
| distilgpt2 | MT_vs_TM | 10.5809              | 1.8042              | 0.8545                        | 0.2156                       | 0.7111      | 0.1725     | 0.2889                | 0.1725               | 80.0000 |
| distilgpt2 | NM_vs_MN | 3.7641               | 0.9414              | 0.3098                        | 0.0893                       | 0.9583      | 0.0283     | 0.0417                | 0.0283               | 80.0000 |
| distilgpt2 | NQ_vs_QN | 5.7897               | 0.6043              | 0.4841                        | 0.1058                       | 0.8844      | 0.0472     | 0.1156                | 0.0472               | 80.0000 |
| distilgpt2 | NT_vs_TN | 11.1703              | 1.8386              | 0.8664                        | 0.1749                       | 0.7046      | 0.1705     | 0.2954                | 0.1705               | 80.0000 |
| distilgpt2 | QM_vs_MQ | 7.7162               | 0.9341              | 0.6474                        | 0.1543                       | 0.8039      | 0.0973     | 0.1961                | 0.0973               | 80.0000 |
| distilgpt2 | QT_vs_TQ | 2.7566               | 0.6003              | 0.3249                        | 0.1427                       | 0.9571      | 0.0470     | 0.0429                | 0.0470               | 80.0000 |
| gpt2       | MT_vs_TM | 11.3260              | 1.9708              | 0.3072                        | 0.0694                       | 0.9707      | 0.0080     | 0.0293                | 0.0080               | 80.0000 |
| gpt2       | NM_vs_MN | 5.2897               | 1.1579              | 0.1894                        | 0.0387                       | 0.9901      | 0.0035     | 0.0099                | 0.0035               | 80.0000 |
| gpt2       | NQ_vs_QN | 11.1700              | 1.5640              | 0.3855                        | 0.0863                       | 0.9337      | 0.0283     | 0.0663                | 0.0283               | 80.0000 |
| gpt2       | NT_vs_TN | 23.9958              | 3.0734              | 0.7100                        | 0.1186                       | 0.9667      | 0.0305     | 0.0333                | 0.0305               | 80.0000 |
| gpt2       | QM_vs_MQ | 13.3556              | 2.9359              | 0.5125                        | 0.1313                       | 0.8888      | 0.0563     | 0.1112                | 0.0563               | 80.0000 |
| gpt2       | QT_vs_TQ | 3.9191               | 0.8893              | 0.2190                        | 0.0846                       | 0.9761      | 0.0196     | 0.0239                | 0.0196               | 80.0000 |

# Semantic equivalence control

| model              | label          | mean_noncommutativity | std_noncommutativity | mean_relative_commutator_std | std_relative_commutator_norm | mean_cosine | std_cosine | n        |
|--------------------|----------------|-----------------------|----------------------|------------------------------|------------------------------|-------------|------------|----------|
| bert-base-uncased  | equivalent     | 0.1130                | 0.0409               | 0.4818                       | 0.0809                       | 0.8870      | 0.0409     | 400.0000 |
| bert-base-uncased  | non_equivalent | 0.3291                | 0.1134               | 0.8065                       | 0.1326                       | 0.6709      | 0.1134     | 400.0000 |
| distilroberta-base | equivalent     | 0.1295                | 0.0483               | 0.5406                       | 0.1277                       | 0.8705      | 0.0483     | 400.0000 |
| distilroberta-base | non_equivalent | 0.2319                | 0.0381               | 0.6868                       | 0.0560                       | 0.7681      | 0.0381     | 400.0000 |
| roberta-base       | equivalent     | 0.1543                | 0.0630               | 0.5831                       | 0.1304                       | 0.8457      | 0.0630     | 400.0000 |
| roberta-base       | non_equivalent | 0.2784                | 0.0439               | 0.7465                       | 0.0597                       | 0.7216      | 0.0439     | 400.0000 |

# Semantic equivalence effect sizes

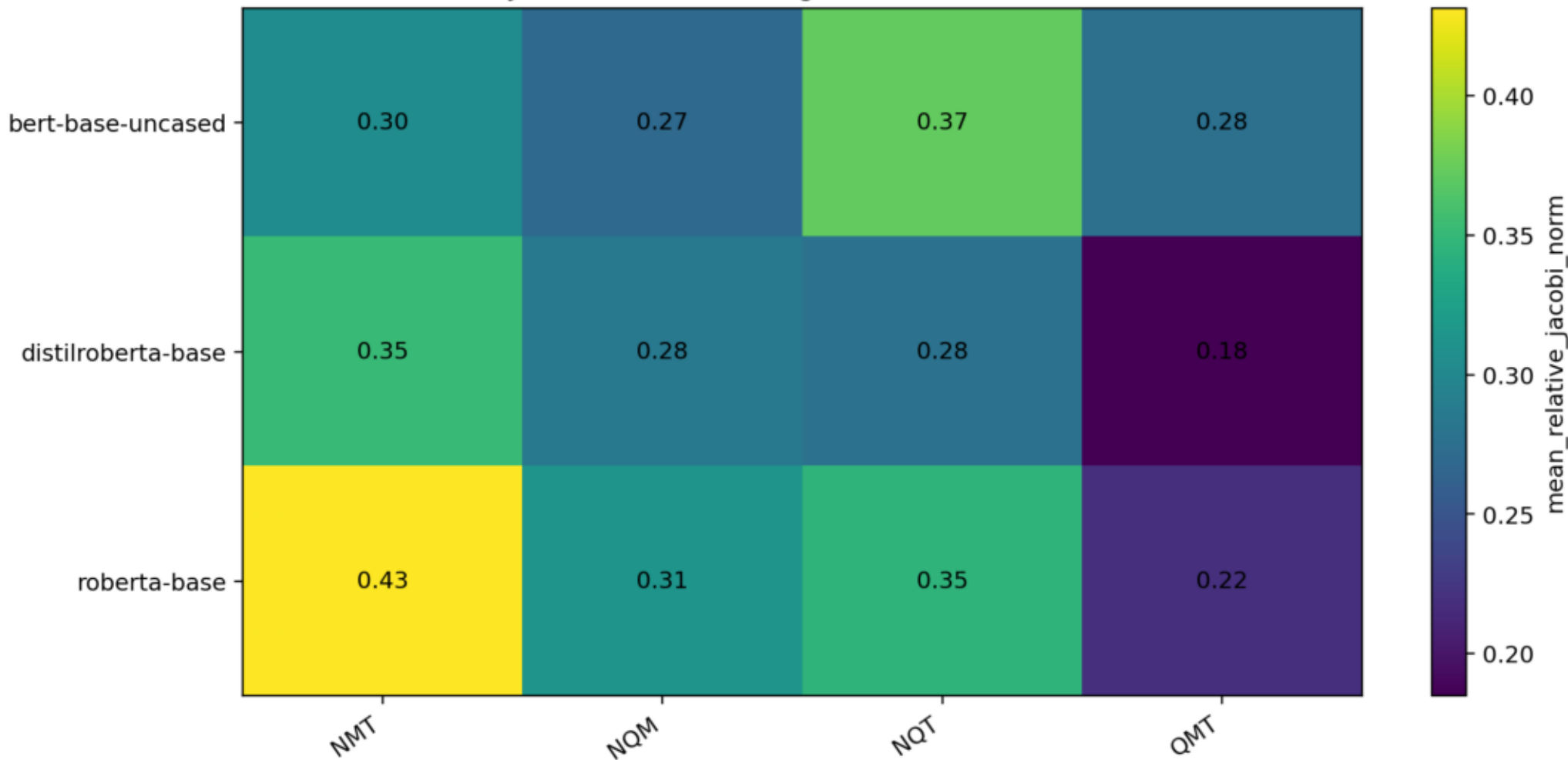
| model              | equiv_mean | non_equiv_mean | diff   | cohens_d | equiv_std | non_equiv_std |
|--------------------|------------|----------------|--------|----------|-----------|---------------|
| bert-base-uncased  | 0.1130     | 0.3291         | 0.2161 | 2.5344   | 0.0409    | 0.1134        |
| distilroberta-base | 0.1295     | 0.2319         | 0.1024 | 2.3536   | 0.0483    | 0.0381        |
| roberta-base       | 0.1543     | 0.2784         | 0.1241 | 2.2854   | 0.0630    | 0.0439        |

# Composition summary: strongest noncommutativity rows

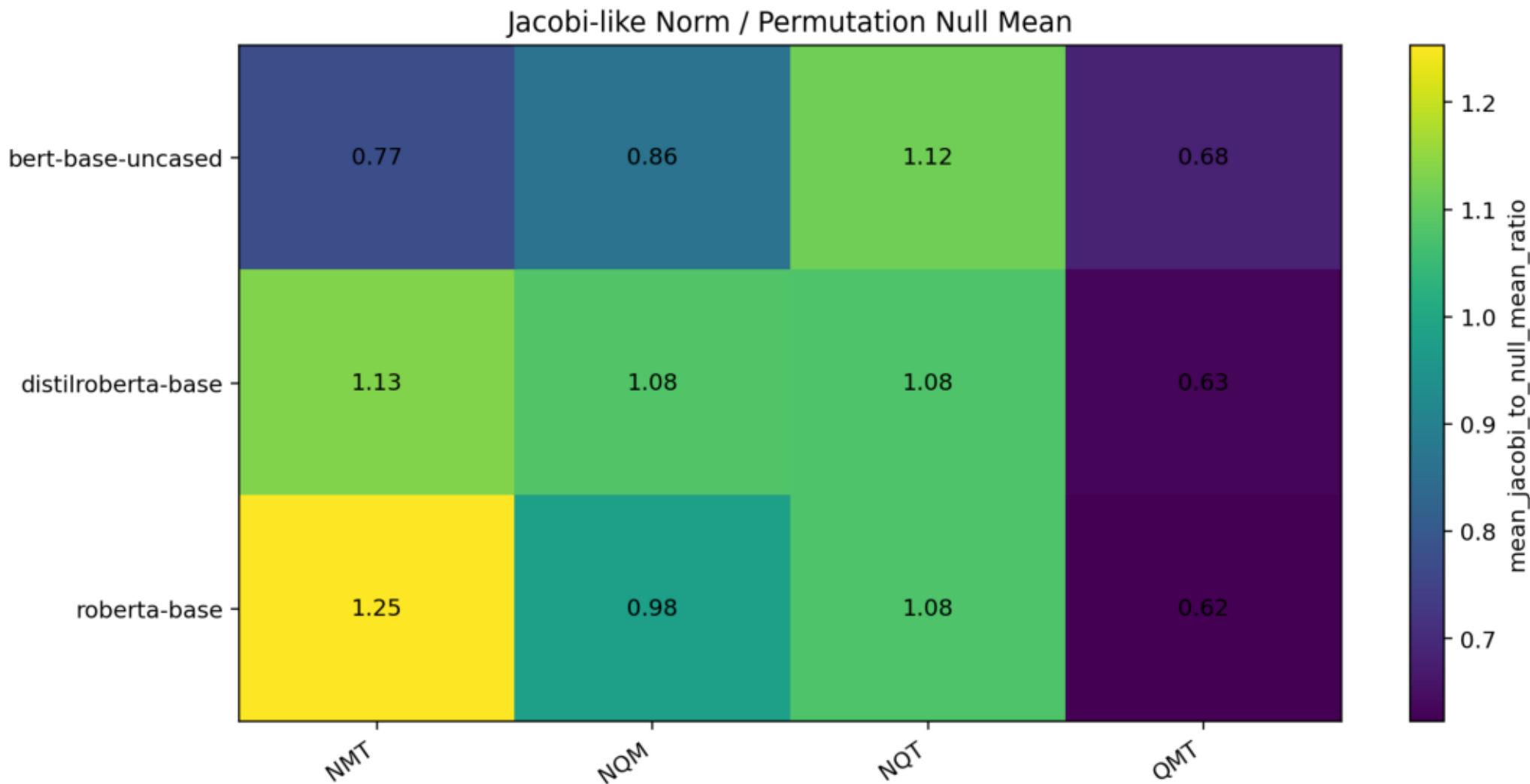
| model              | pair     | mean_commutator_norm | std_commutator_norm | mean_relative_commutator_norm | std_relative_commutator_norm | mean_cosine | std_cosine | mean_noncommutativity | std_noncommutativity | n       |
|--------------------|----------|----------------------|---------------------|-------------------------------|------------------------------|-------------|------------|-----------------------|----------------------|---------|
| bert-base-uncased  | NT_vs_TN | 5.1565               | 0.5832              | 0.9908                        | 0.1066                       | 0.5102      | 0.1009     | 0.4898                | 0.1009               | 80.0000 |
| roberta-base       | NQ_vs_QN | 1.6422               | 0.0726              | 0.7846                        | 0.0364                       | 0.6928      | 0.0279     | 0.3072                | 0.0279               | 80.0000 |
| bert-base-uncased  | MT_vs_TM | 4.2726               | 0.3075              | 0.7817                        | 0.0954                       | 0.6938      | 0.0727     | 0.3062                | 0.0727               | 80.0000 |
| roberta-base       | QM_vs_MQ | 1.5649               | 0.0863              | 0.7714                        | 0.0541                       | 0.7032      | 0.0410     | 0.2968                | 0.0410               | 80.0000 |
| distilroberta-base | MT_vs_TM | 1.3504               | 0.0606              | 0.7683                        | 0.0387                       | 0.7124      | 0.0302     | 0.2876                | 0.0302               | 80.0000 |
| bert-base-uncased  | QM_vs_MQ | 4.4793               | 0.2156              | 0.7341                        | 0.0487                       | 0.7340      | 0.0347     | 0.2660                | 0.0347               | 80.0000 |
| roberta-base       | MT_vs_TM | 1.3956               | 0.0770              | 0.7215                        | 0.0560                       | 0.7411      | 0.0394     | 0.2589                | 0.0394               | 80.0000 |
| roberta-base       | NT_vs_TN | 1.4185               | 0.1071              | 0.7172                        | 0.0599                       | 0.7429      | 0.0428     | 0.2571                | 0.0428               | 80.0000 |
| bert-base-uncased  | NQ_vs_QN | 4.0903               | 0.2939              | 0.6970                        | 0.0493                       | 0.7598      | 0.0347     | 0.2402                | 0.0347               | 80.0000 |
| distilroberta-base | NQ_vs_QN | 1.4541               | 0.0644              | 0.6731                        | 0.0384                       | 0.7735      | 0.0257     | 0.2265                | 0.0257               | 80.0000 |
| distilroberta-base | NT_vs_TN | 1.3975               | 0.0725              | 0.6675                        | 0.0342                       | 0.7783      | 0.0232     | 0.2217                | 0.0232               | 80.0000 |
| distilroberta-base | QM_vs_MQ | 1.3672               | 0.0842              | 0.6613                        | 0.0460                       | 0.7926      | 0.0287     | 0.2074                | 0.0287               | 80.0000 |
| bert-base-uncased  | QT_vs_TQ | 3.1001               | 0.1729              | 0.5463                        | 0.0440                       | 0.8513      | 0.0231     | 0.1487                | 0.0231               | 80.0000 |
| roberta-base       | QT_vs_TQ | 0.7784               | 0.0701              | 0.3860                        | 0.0342                       | 0.9269      | 0.0128     | 0.0731                | 0.0128               | 80.0000 |
| distilroberta-base | QT_vs_TQ | 0.6580               | 0.0281              | 0.3422                        | 0.0173                       | 0.9415      | 0.0059     | 0.0585                | 0.0059               | 80.0000 |
| bert-base-uncased  | NM_vs_MN | 1.2527               | 0.2368              | 0.2305                        | 0.0538                       | 0.9728      | 0.0125     | 0.0272                | 0.0125               | 80.0000 |
| roberta-base       | NM_vs_MN | 0.3865               | 0.0403              | 0.2032                        | 0.0248                       | 0.9794      | 0.0050     | 0.0206                | 0.0050               | 80.0000 |
| distilroberta-base | NM_vs_MN | 0.3128               | 0.0212              | 0.1721                        | 0.0201                       | 0.9852      | 0.0036     | 0.0148                | 0.0036               | 80.0000 |

# Signed permutation relative norm

Jacobi-like Alternating Sum Relative Norm

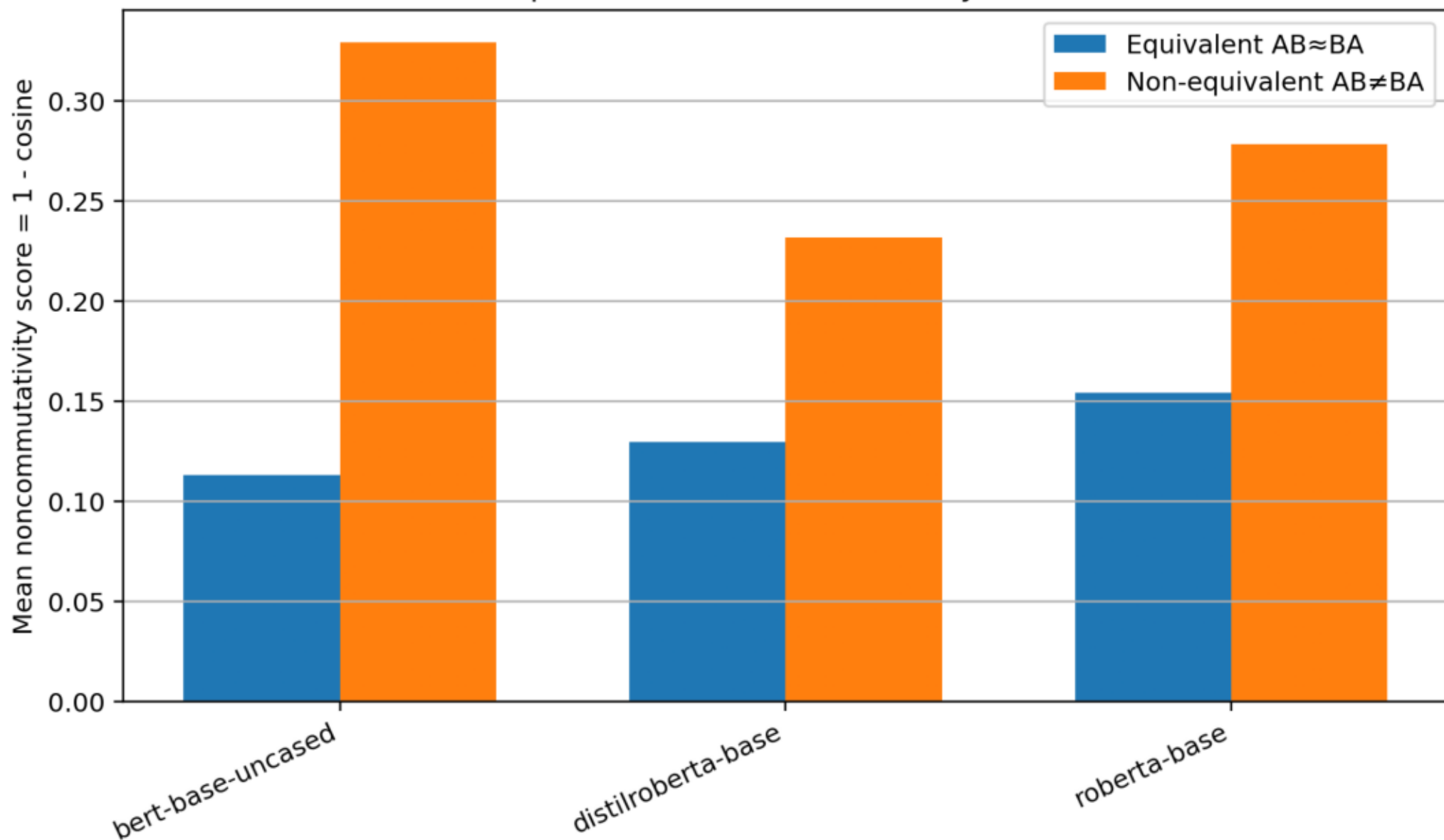


# Signed permutation norm versus permutation-null

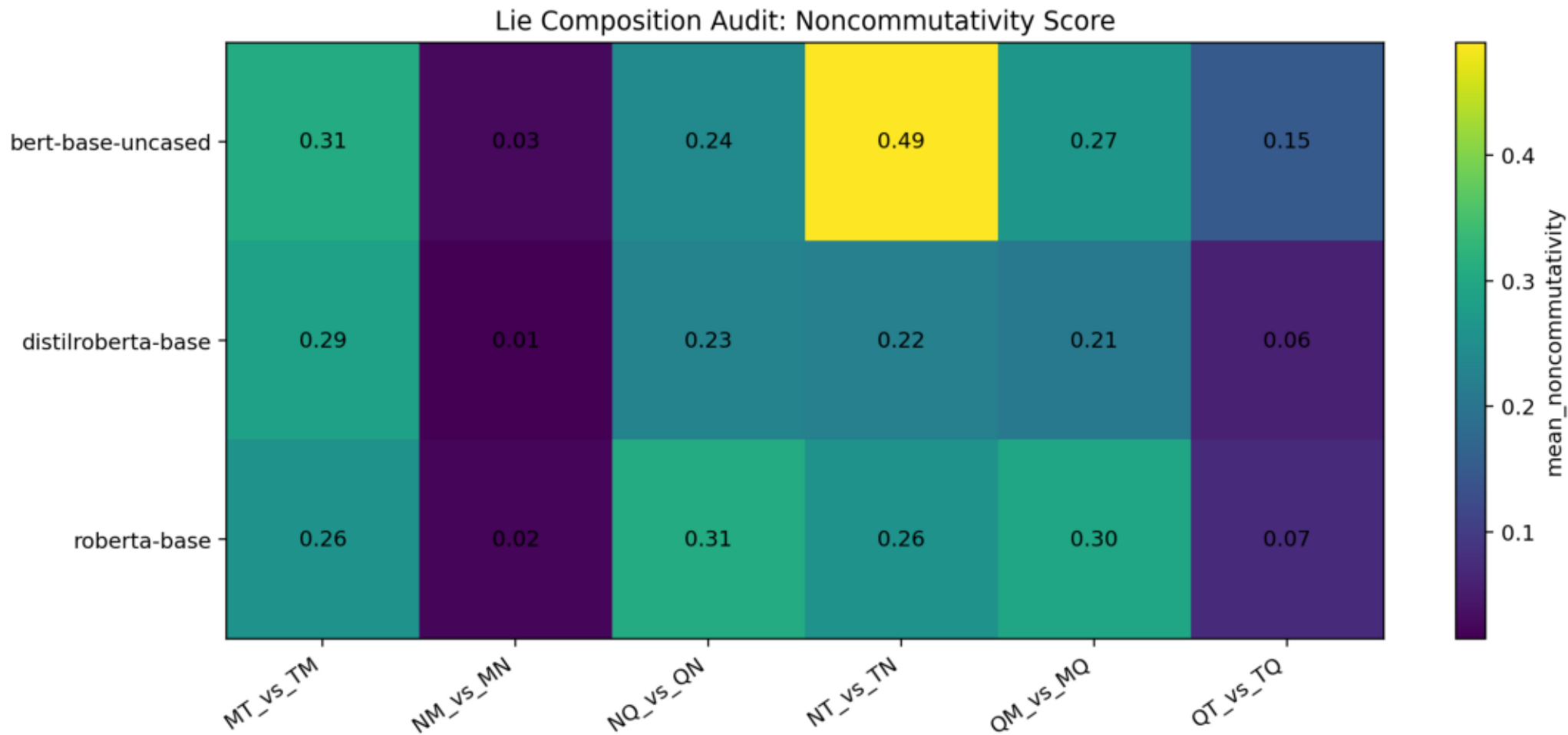


# Semantic equivalent vs non-equivalent control

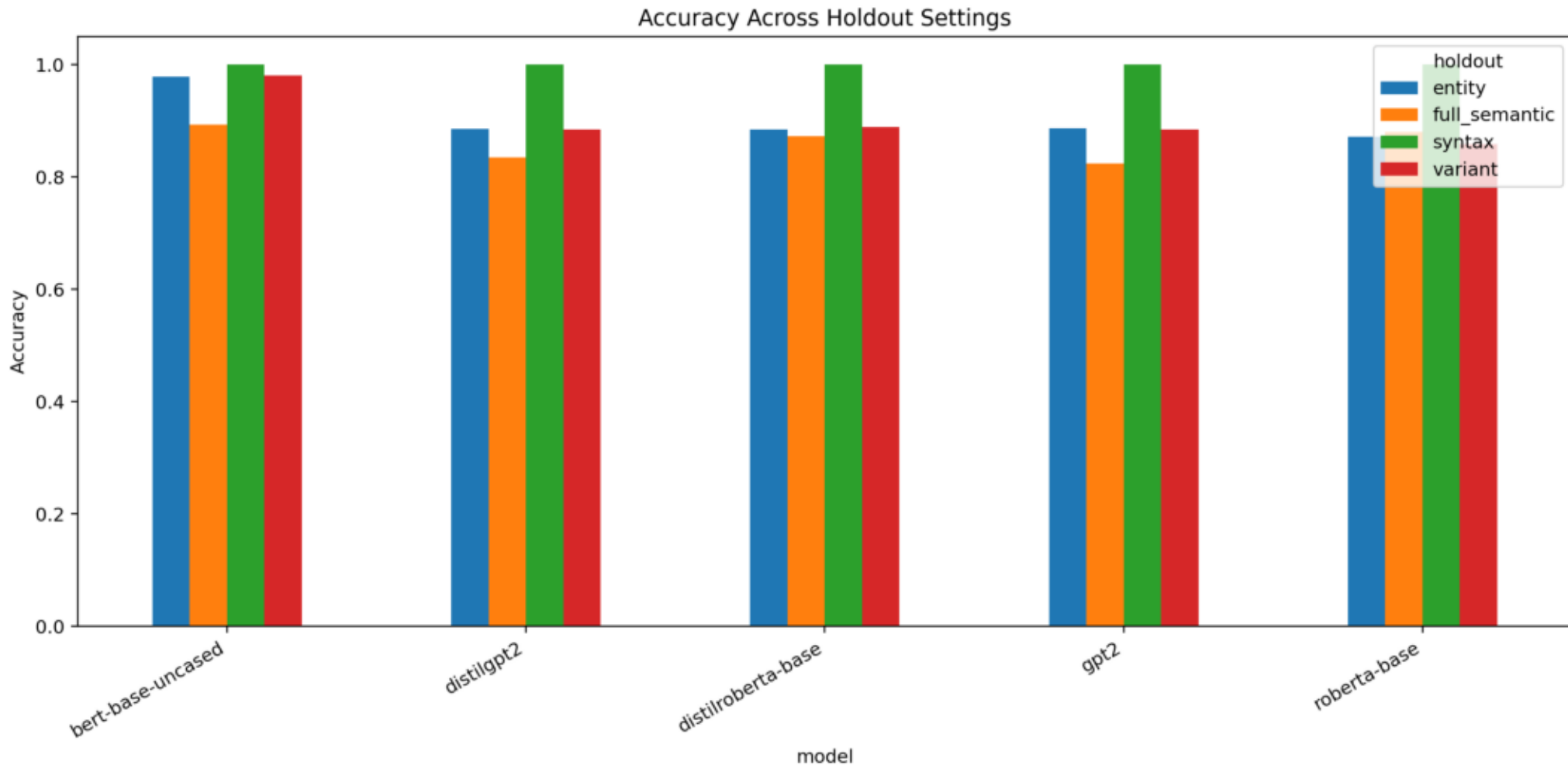
Semantic Equivalence Control for Lie-Style Commutators



# Composition noncommutativity heatmap

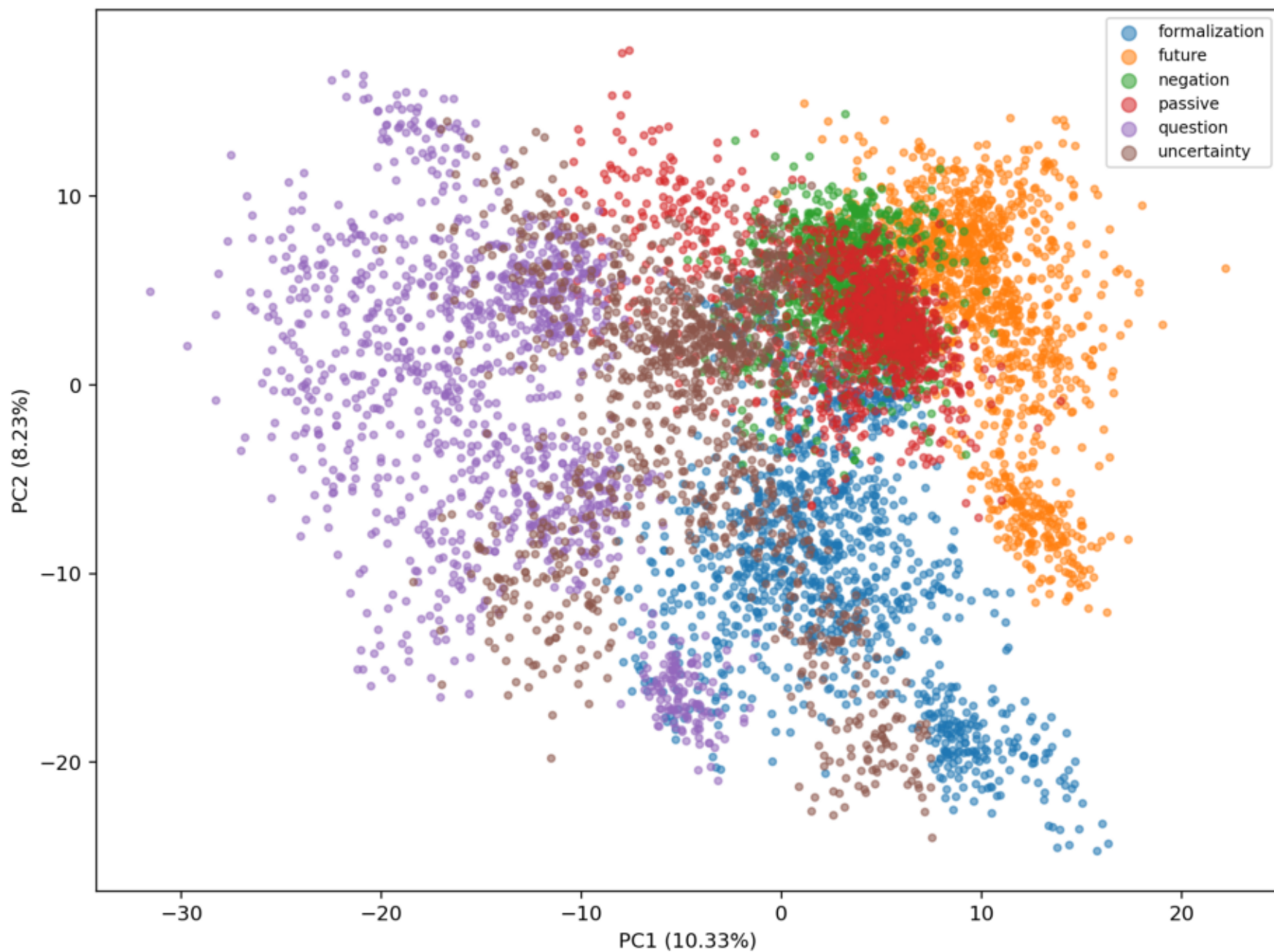


# Original holdout accuracy comparison



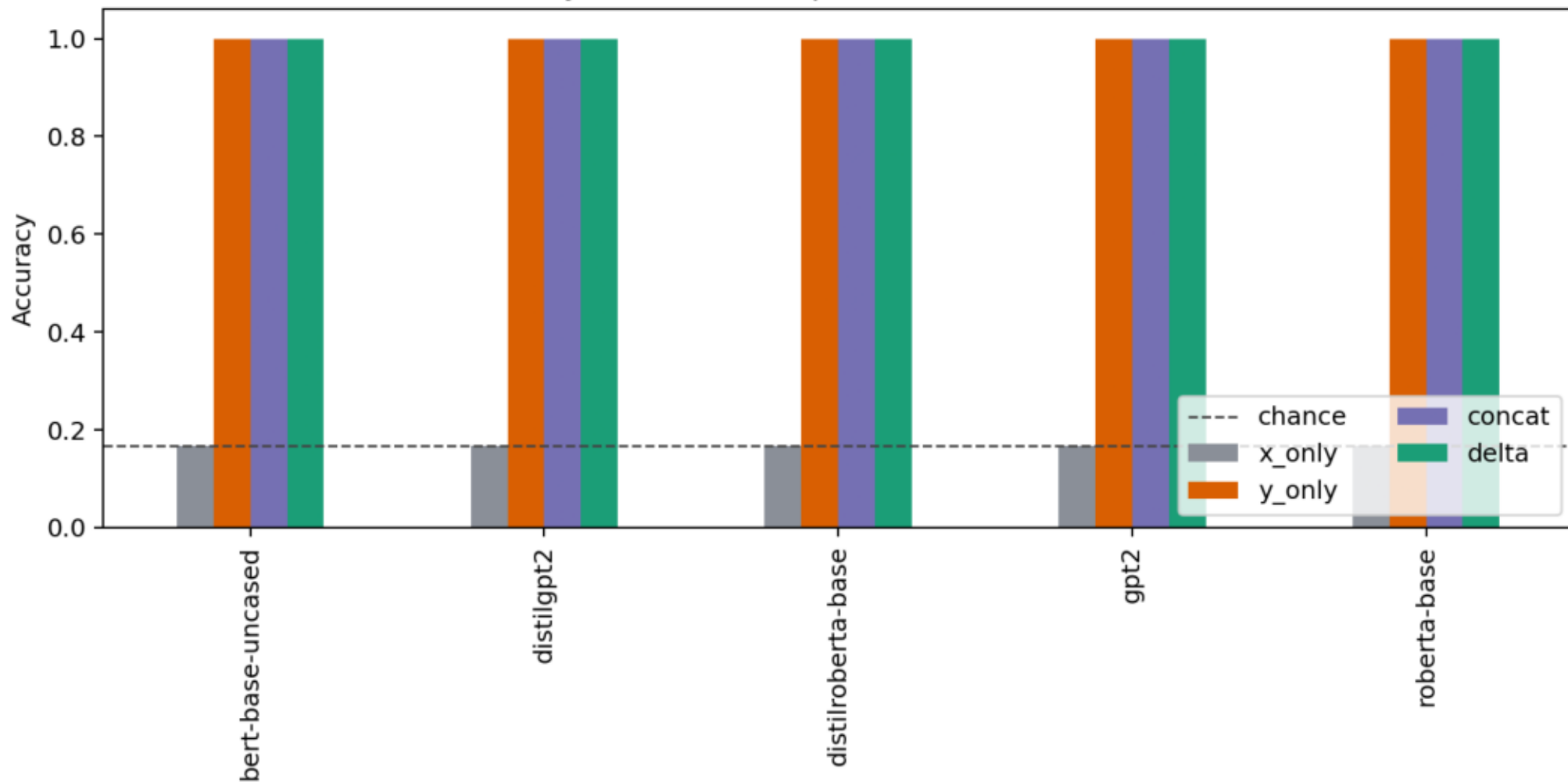
# Original PCA class geometry

PCA of Delta Vectors: bert-base-uncased



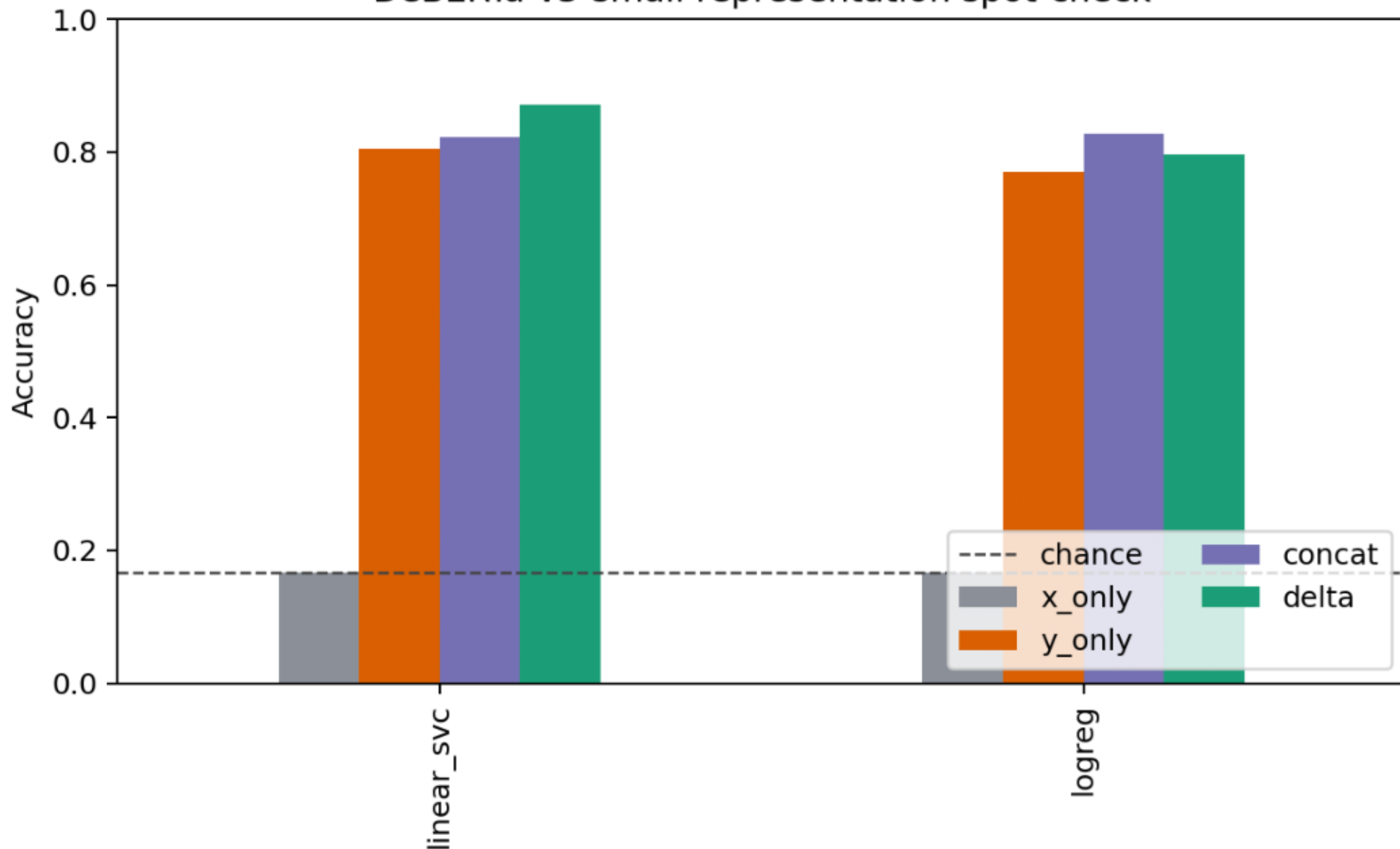
# Syntax representation ablation

Syntax holdout representation ablation

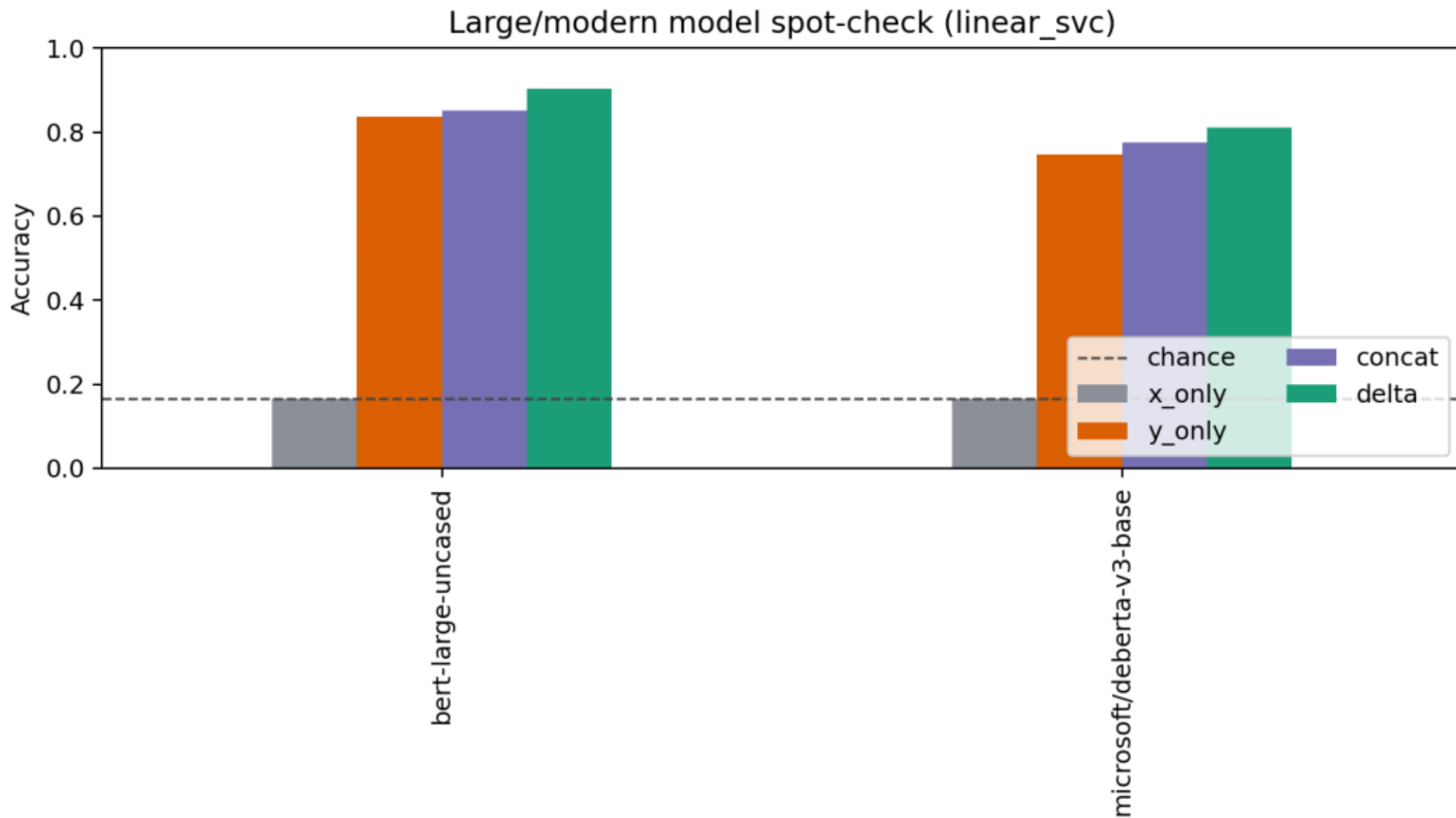


# DeBERTa-v3-small spot-check

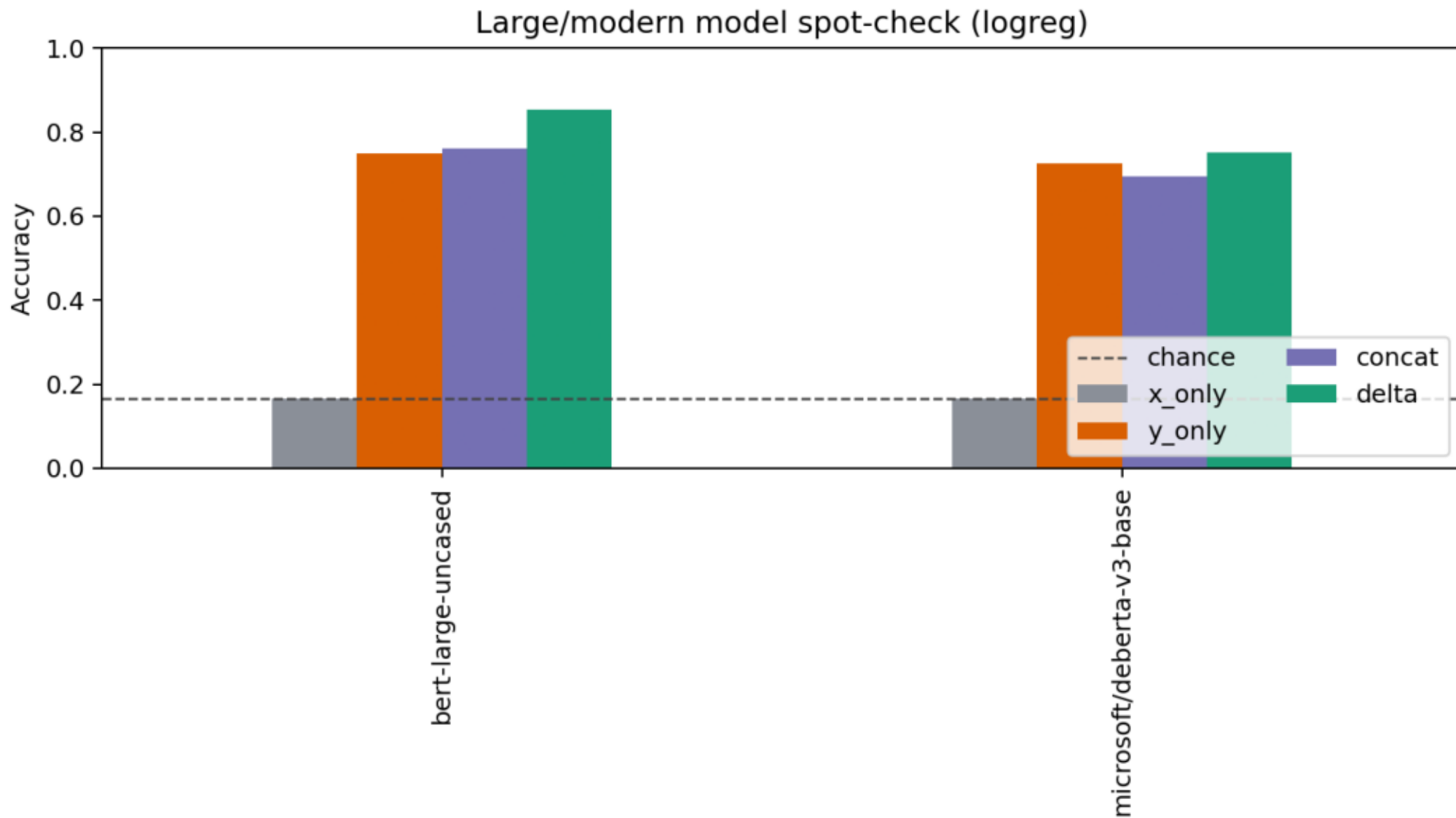
DeBERTa-v3-small representation spot-check



## Large/modern spot-check: Linear SVC

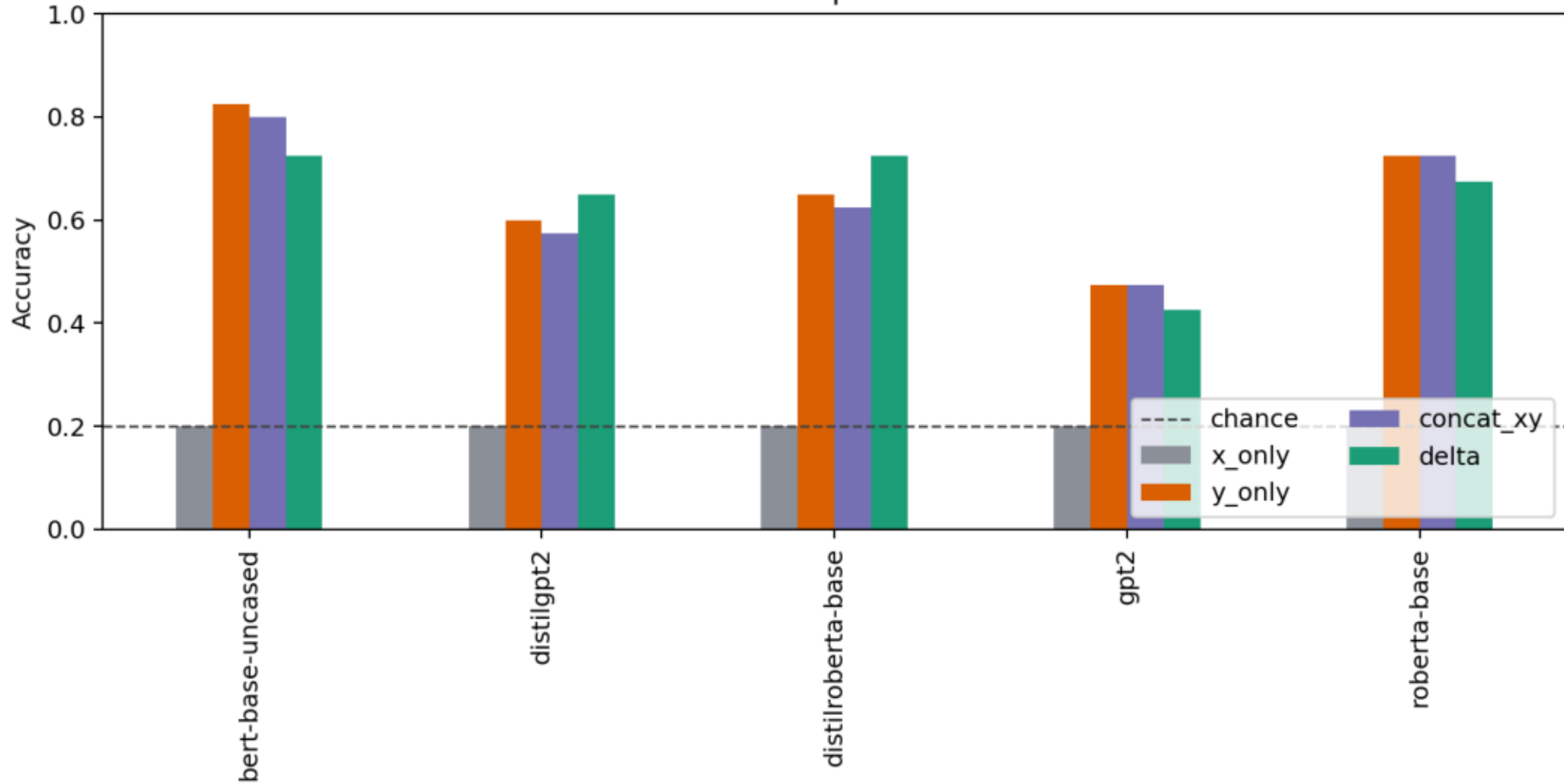


# Large/modern spot-check: logistic regression

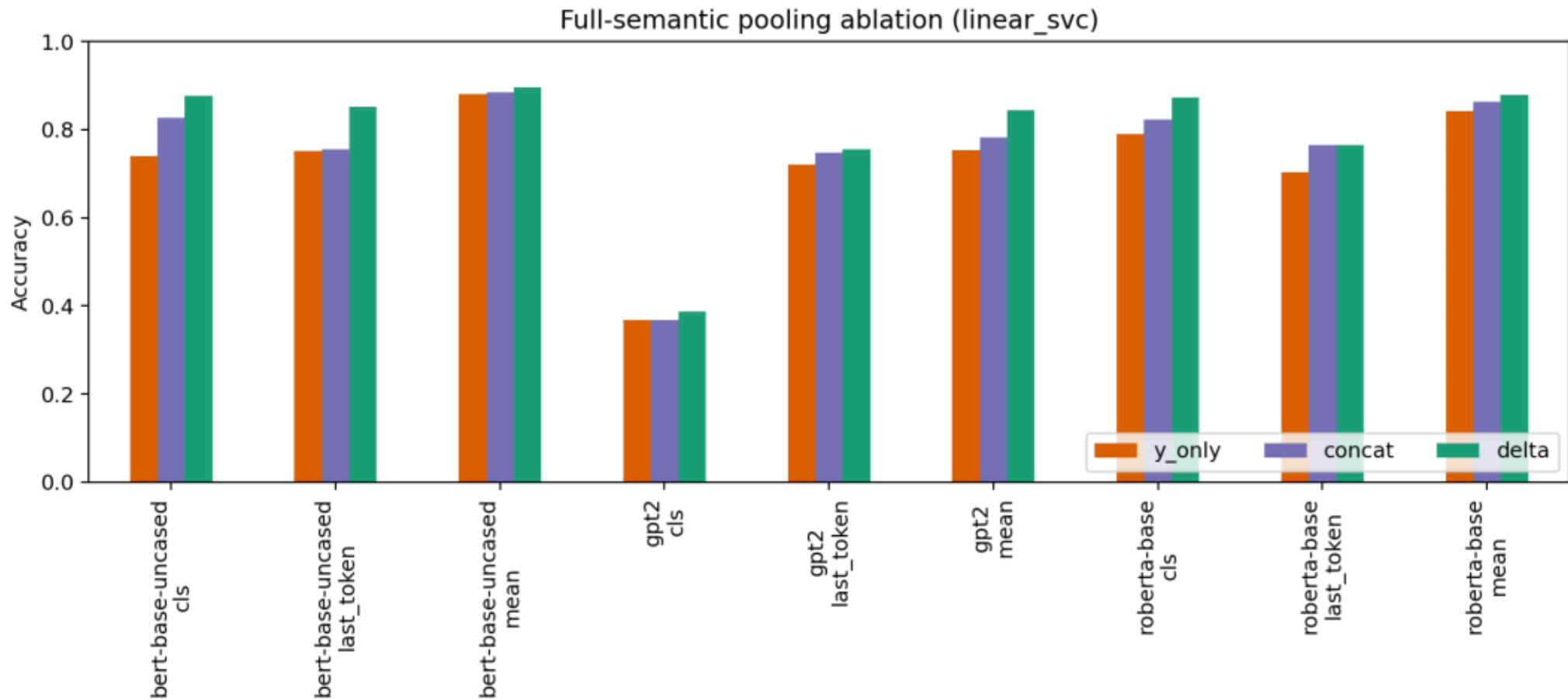


# UPAT hard-holdout ablation

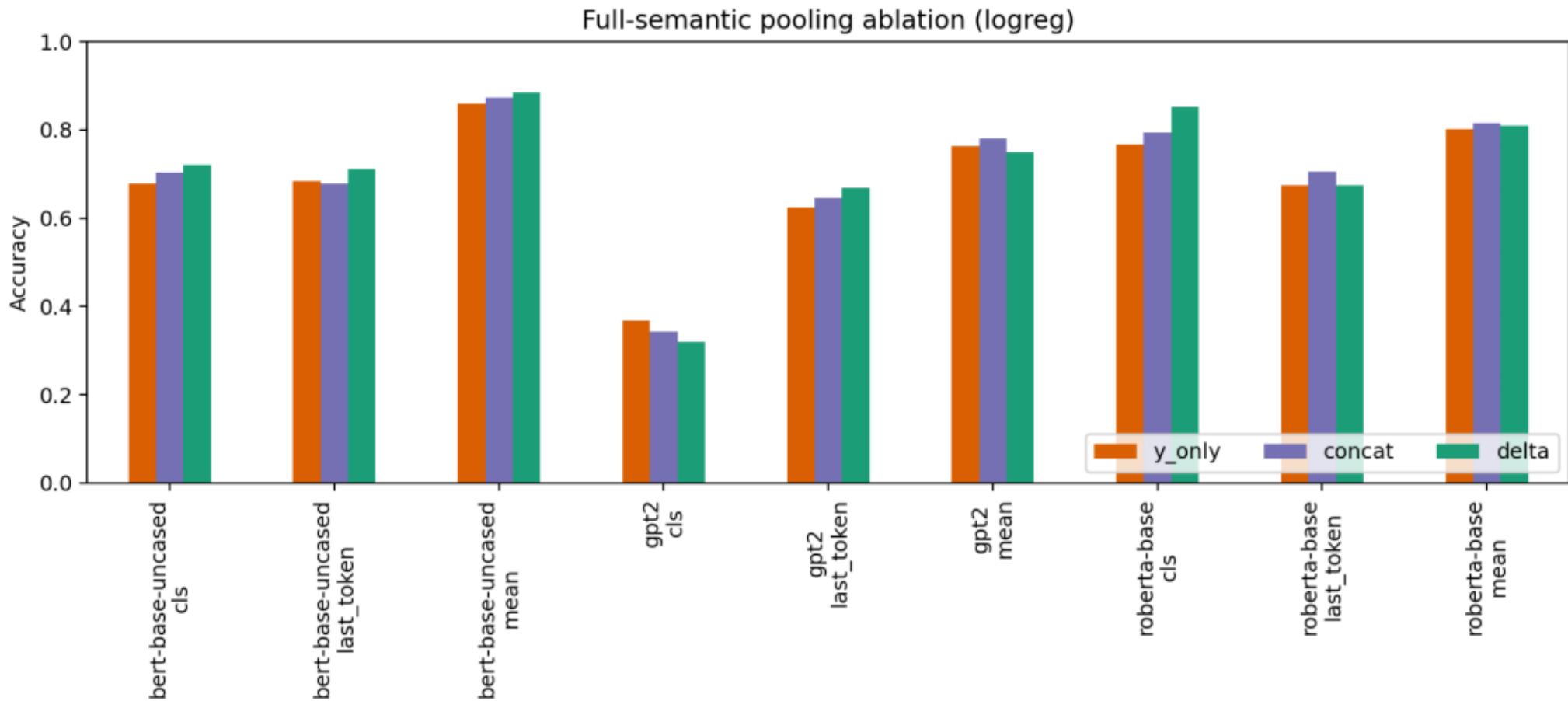
UPAT hard-holdout representation ablation



# Full-semantic pooling ablation: Linear SVC

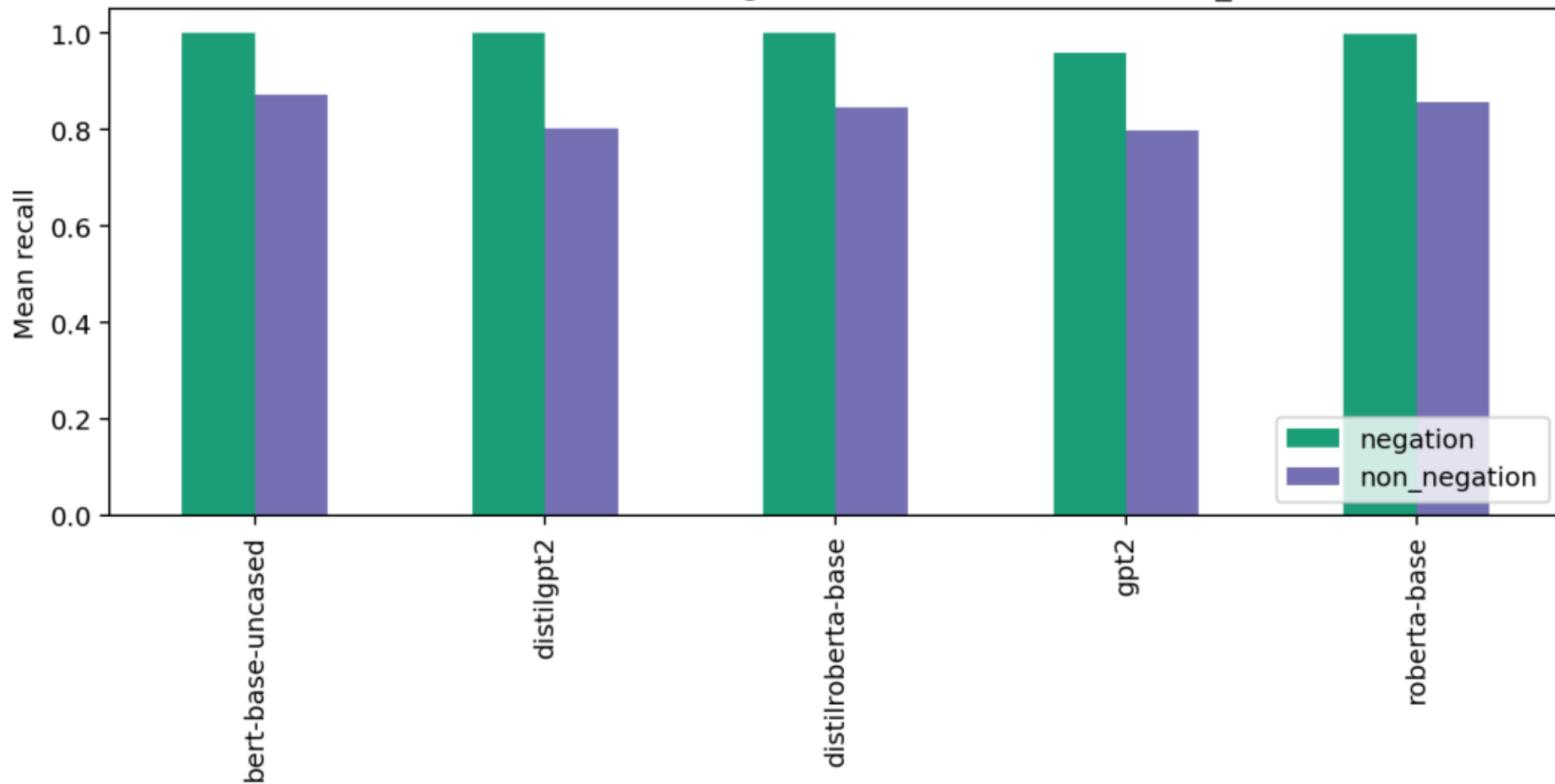


# Full-semantic pooling ablation: logistic regression



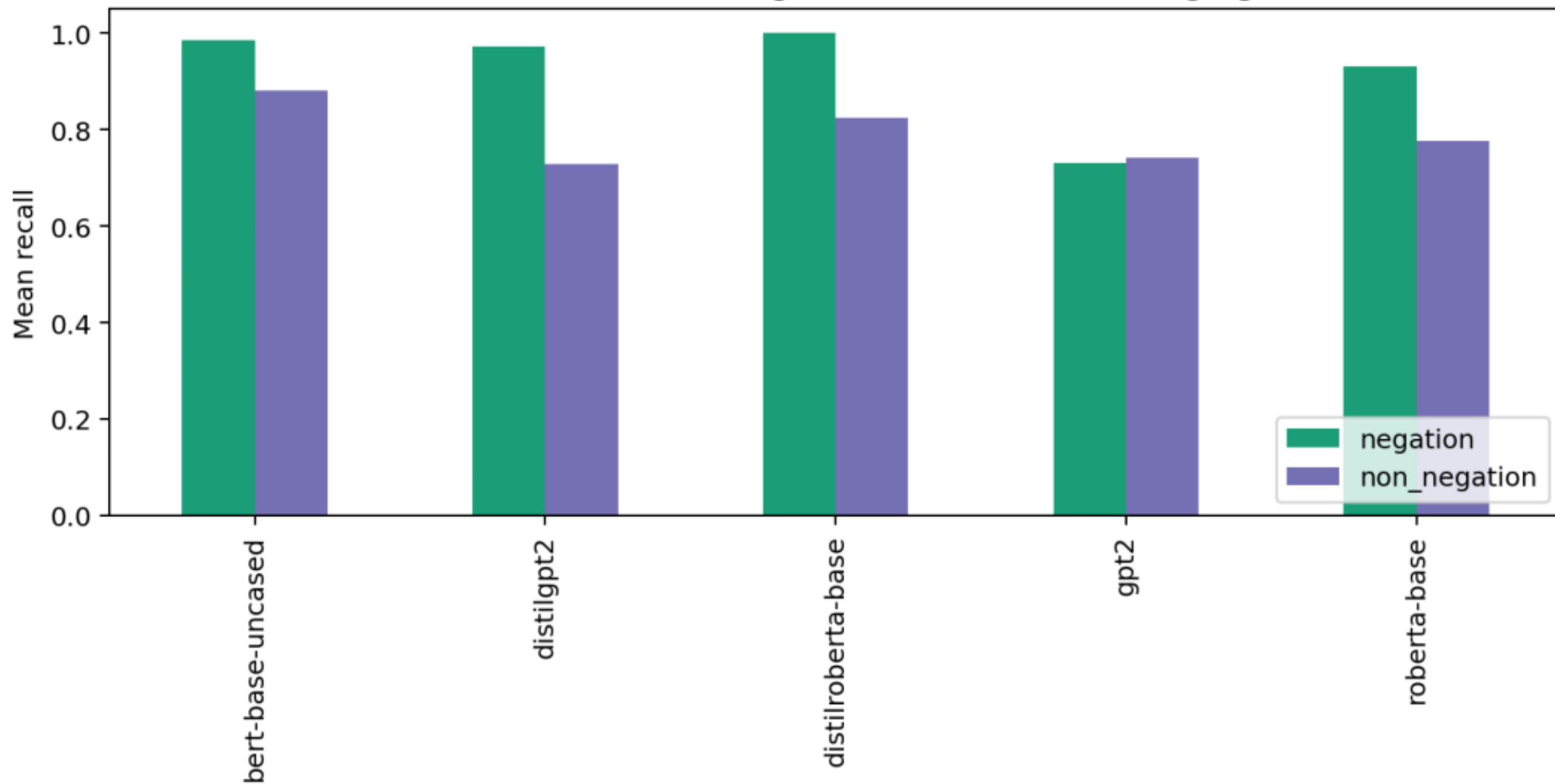
# Confusion analysis: negation recall Linear SVC

Full-semantic recall: negation vs other classes (linear\_svc)



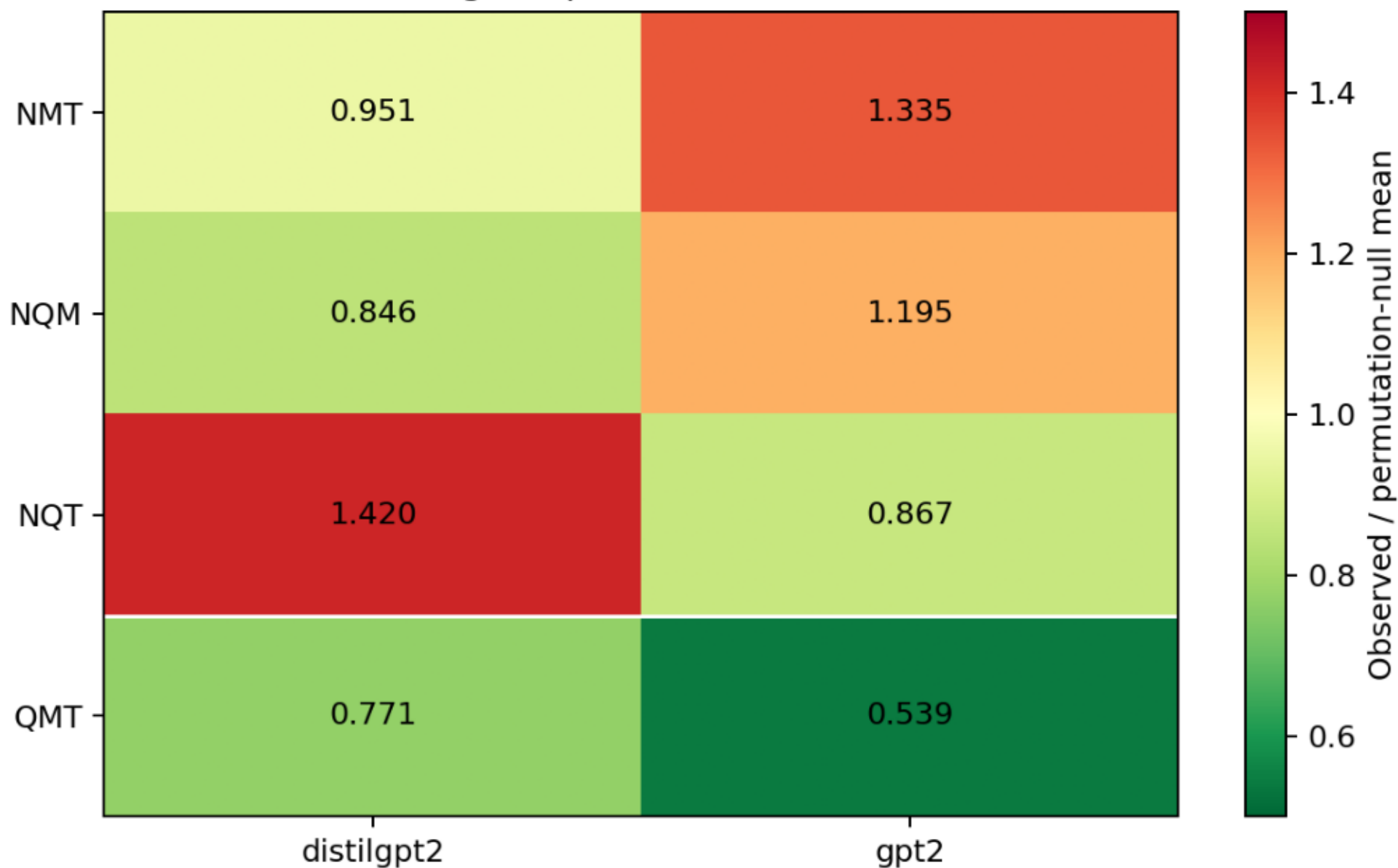
# Confusion analysis: negation recall logistic regression

Full-semantic recall: negation vs other classes (logreg)

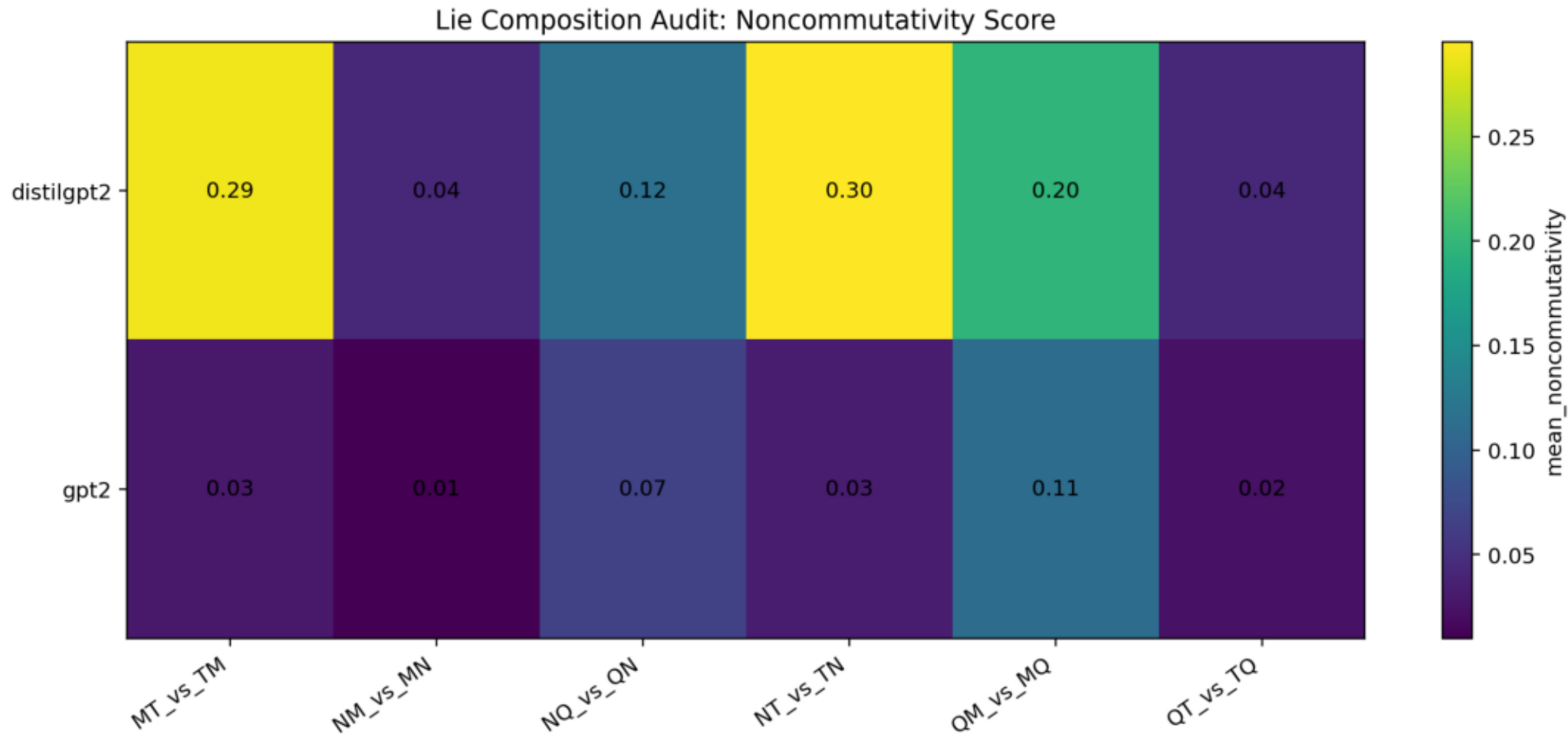


# Decoder signed permutation replication

Decoder signed permutation ratio to null



# Decoder pairwise composition



# Audit notes for external verifier

Antisymmetry is not reported as evidence because the implementation defines [B,A] as the negative order of [A,B]. It is a tautological implementation check only.

A literal nested-commutator Jacobi expression over the same six endpoint vectors cancels algebraically. The reported third-order diagnostic is the non-tautological signed permutation composition sum  $ABC+BCA+CAB-ACB-CBA-BAC$ .

Duplicate endpoint templates were detected and removed before finalizing today's result. The final dataset has 400 Jacobi rows and zero duplicate endpoint sets.

New Track 1 result: syntax  $y_{\text{only}}=1.0$  confirms target/surface leakage for the syntax split. New Track 1 spot-check: DeBERTa-v3-small supports delta superiority for Linear SVC but not for logistic regression. New Track 2 result: GPT-2 and DistilGPT-2 both keep QMT below permutation null, while negation triples remain mixed.

2026-06-13 update: after cache cleanup, BERT-large and DeBERTa-v3-base were both run successfully. Delta is best for both Linear SVC and logistic regression on both models.

Second 2026-06-13 update: UPAT is now explicitly reported as a hard-holdout boundary condition. Full-semantic pooling ablation, confusion/negation analysis, decoder pairwise composition, and signed-permutation multiple-testing correction are included.

Remaining blockers: resolve or expand UPAT, representation-ablation tables for every non-syntax holdout split, grammar-generated templates, endpoint-only controls for Track 2, composition layer/pooling ablations, and final bibliography formatting.

# Round-3 Reviewer Response

# Reviewer Response Round 3

Date: 2026-06-13

This note records the current response to the latest review of the research package.

## Fixed in the Current Drafts

### Antisymmetry

The Track 2 draft no longer presents antisymmetry as a discovery. It is now described as a tautological implementation check:

```
```text
[A,B] = delta_AB - delta_BA
[B,A] = delta_BA - delta_AB
```
```

Therefore  $[A,B] = -[B,A]$  follows for arbitrary vectors and does not provide evidence about transformer representations.

### UPAT and y\_only Confounding

The Track 1 draft now explicitly reports UPAT as a hard-holdout boundary result. UPAT sometimes favors  $y\_only$  over  $\delta$ , and no UPAT  $\delta$  vs  $y\_only$  McNemar comparison is significant. The claim is narrowed:

> Delta can add useful relational information, but endpoint-only features remain a serious confounder and can dominate under smaller, harder holdouts.

### Layer-0 Syntax Result

The layer-0 syntax result is now framed as a red flag. Perfect separability at the embedding layer indicates token/form cues rather than deep semantic geometry.

### Synthetic Lie Templates

The Track 2 draft now explicitly states that the current Lie-style composition dataset is synthetic and contains stable lexical markers. The result is a controlled diagnostic, not a general natural-language operator algebra.

### ### Mixed Signed-Permutation Results

The Track 2 draft now explains that several triples are below null in some models, while others are above null. `QMT` is promoted only because it is the only tested triple that is below null across all five models after table-level correction.

### ## Still Required Before Submission

1. Add a Procrustes alignment null baseline for UPAT:
  - random-label or random-pairing alignment
  - report null distribution for F1 gain
2. Increase shuffle controls from 100 permutations to at least 1000 before reporting precise p-values.
3. Add null baselines for pairwise commutator norms  $||[A,B]||$ .
4. Replace hand-written Lie templates with grammar-generated paraphrase templates.
5. Run endpoint-only controls for composition endpoints.
6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.

### ## Current Submission Readiness

The package is now stronger as a research record, but it should not be submitted as a main-track paper yet. The best current framing is:

- Track 1: a representation-geometry study with clear endpoint confounders and a hard-holdout boundary.
- Track 2: a diagnostic study of local signed-permutation coherence, with `QMT` as the only cross-model stable triple and negation as a composition instability.

# Updated Research Roadmap

# Research Program: From Linguistic Transformation Vectors to Composition Diagnostics

## Core question

Can linguistic transformations be represented not only as separable classes in transformer embedding spaces, but as reusable geometric operations with meaningful composition structure?

The project currently has two paper tracks.

## Strategic narrative

The strongest long-term story is broader than either current draft:

> Transformers may encode linguistic operators as geometric objects in a low-dimensional, partially universal transformation subspace, and this subspace may be transferable across architectures and languages.

Current results are not enough to claim this yet. The Procrustes transfer numbers are promising, but they remain exploratory until null baselines are added. The roadmap below treats cross-model and cross-lingual transfer as the next central hypothesis, not as a finished conclusion.

## Track 1: Geometric transformation vectors

Working title:

**Geometric Transformation Vectors in Transformer Embedding Spaces**

Central claim:

Transformer embedding spaces contain displacement directions that encode linguistic transformations. These directions are recoverable across multiple models and holdout regimes.

Current evidence:

- Delta vectors classify transformations well across BERT, RoBERTa, DistilRoBERTa, GPT-2, and DistilGPT-2.
- Syntax holdouts reach `1.0` accuracy across tested models, but the new representation ablation shows `y\_only=1.0` as well. This is now interpreted as endpoint/surface leakage, not as deep generalization.
- Full semantic holdouts remain above chance, with accuracies around `0.82-0.89`.
- Entity and variant holdouts remain strong, especially for BERT-family models.

- Modern/larger spot-checks now support the main delta advantage:
  - BERT-large Linear SVC:  $\delta=0.903$ ,  $\text{concat}=0.851$ ,  $y_{\text{only}}=0.838$
  - BERT-large logistic regression:  $\delta=0.854$ ,  $\text{concat}=0.760$ ,  $y_{\text{only}}=0.750$
  - DeBERTa-v3-base Linear SVC:  $\delta=0.812$ ,  $\text{concat}=0.776$ ,  $y_{\text{only}}=0.747$
  - DeBERTa-v3-base logistic regression:  $\delta=0.752$ ,  $\text{concat}=0.694$ ,  $y_{\text{only}}=0.726$
- UPAT hard-holdout is a boundary result:  $\delta$  is worse than  $y_{\text{only}}$  for BERT, RoBERTa, and GPT-2, and no UPAT delta-vs-y McNemar test is significant.
- Full-semantic pooling ablation now supports mean pooling as a defensible main choice, while showing that the decoder effect is partly pooling-dependent.

Scientific status:

This is the more mature paper. It can be written as a representation-geometry result with careful controls against template memorization.

Main risk:

Some signal may come from target-sentence artifacts rather than pure transformation geometry. The concrete high-risk case is now confirmed:  $\text{syntax}=1.0$  reproduces under  $y_{\text{only}}$ , so it must be interpreted as a target/surface artifact unless future controls overturn that. UPAT also shows that  $\delta > y_{\text{only}}$  is not universal under small hard-holdout regimes. The paper needs strong source-only, target-only, delta-only, and paraphrase controls.

## ## Track 2: Signed permutation coherence for linguistic operators

Working title:

**Third-Order Signed Permutation Coherence for Linguistic Transformations in Transformer Embedding Spaces**

Central claim:

Some linguistic transformations behave like locally noncommuting operations in embedding space. Certain triples show third-order signed permutation cancellation stronger than permutation-null baselines, but the evidence supports local composition structure rather than a global Lie algebra.

Current evidence:

- Composition order matters:  $AB$  and  $BA$  often differ substantially.
- Semantic equivalence controls show lower noncommutativity for equivalent pairs than for non-equivalent pairs.
- Antisymmetry is not treated as evidence. It is a tautological implementation check because  $[A,B] = \delta_{AB} - \delta_{BA}$  implies  $[B,A] = -[A,B]$ .
- The non-tautological third-order signed permutation diagnostic is:

```
```text
S(A,B,C) = ABC + BCA + CAB - ACB - CBA - BAC
```
```

- The `QMT` triple shows robust cancellation across tested models:
  - BERT: ratio to permutation null `0.683`, CI `[0.677, 0.689]`
  - DistilRoBERTa: ratio `0.631`, CI `[0.627, 0.636]`
  - RoBERTa: ratio `0.623`, CI `[0.617, 0.629]`
- Decoder replication is now partially complete:
  - GPT-2 `QMT`: ratio `0.539`, CI `[0.519, 0.561]`
  - DistilGPT-2 `QMT`: ratio `0.771`, CI `[0.745, 0.797]`
  - Negation-containing triples remain mixed and model-dependent.
- Decoder pairwise composition summaries are now added, not only decoder third-order summaries.
- Multiple-testing correction over `4 triples x 5 models` supports the narrowed claim: `QMT` is the only below-null triple passing across all five tested models.

Scientific status:

This is promising but more fragile. It should be framed as a null-controlled diagnostic, not as proof of a Lie algebra or as a formal Jacobi identity.

Main risk:

Hand-written templates may induce or suppress cancellation. GPT-2/DistilGPT-2 now support `QMT` below-null cancellation, but negation behaves inconsistently across architectures. The current Lie-style templates still contain stable lexical markers, so the next experiments need grammar-driven templates, endpoint-only controls, commutator null baselines, and a focused negation analysis before expanding the operator set.

## Track 3: Cross-model transformation transfer

Working title:

**\*\*Universal Transformation Subspaces Across Transformer Architectures\*\***

Central hypothesis:

Transformation geometry may be partially model-independent after low-dimensional alignment. A classifier trained on transformation deltas in one model may transfer to another model after Procrustes alignment.

Promising exploratory evidence:

- BERT to `all-mpnet-base-v2`: raw F1 around `0.15`, aligned F1 around `0.857`.
- BERT to `all-MiniLM-L6-v2`: raw F1 around `0.17`, aligned F1 around `0.754`.
- UPAT-large cross-model results show large aligned-transfer gains across several architectures.

Current status:

This could become the strongest paper if it survives null controls. It is not yet promoted because Procrustes has many degrees of freedom and can overfit small alignment sets.

Required gates before promotion:

1. Random-label or random-pairing Procrustes null baseline.
2. Alignment-size curve with held-out evaluation.
3. At least 1000 shuffle/permutation runs for precise p-values.
4. Reverse-direction transfer tests, e.g. small model to large model and large model to small model.
5. Stronger architectures if feasible, such as Llama/Mistral-class embedding spaces or high-quality sentence encoders.

## Track 4: Transformation vectors as editors

Working title:

**\*\*From Transformation Geometry to Controllable Linguistic Editing\*\***

Central hypothesis:

If transformation directions are real, they should not only classify transformations; they should also steer generation or hidden states toward those transformations.

Minimal experiment:

1. Use GPT-2 as the first testbed.
2. Compute a centroid delta for a transformation such as negation or question formation.
3. Inject the vector into the residual stream during generation.
4. Measure whether outputs systematically acquire the target transformation.
5. Compare against random-vector, norm-matched, and wrong-class controls.

Scientific payoff:

This would move the project from descriptive geometry to causal intervention. A positive result would connect the work to representation steering and controllable generation.

## Track 5: Cross-lingual transformation geometry

Working title:

**\*\*Language-Invariant Geometry of Linguistic Transformations\*\***

Central hypothesis:

If transformation geometry is universal rather than English-template-specific, aligned multilingual models should show comparable transformation subspaces across languages.

Minimal experiment:

1. Use mBERT or XLM-R.
2. Build parallel transformation pairs in English and one or more non-English languages.
3. Compare delta geometry within each language.
4. Align language-specific transformation spaces with Procrustes.
5. Test whether classifiers or centroids transfer across languages.

Scientific payoff:

This is the cleanest answer to the criticism that the current effects may be English grammar or template artifacts.

**## Track 6: Dimensionality of transformation manifolds**

Working title:

**\*\*Effective Dimensionality of Linguistic Transformation Subspaces\*\***

Central hypothesis:

Each transformation may occupy a low-dimensional subspace rather than a single direction or the full embedding space.

Measurements:

- PCA spectra of per-class delta matrices.
- Participation ratio of singular values.
- Classification accuracy as a function of retained dimensions.
- Sample-complexity curves for learning each transformation.

Scientific payoff:

This would quantify the complexity of each linguistic operator and explain why capacity curves keep improving with more examples.

## ## Iterative logic

The research direction is:

1. Show transformations are encoded as recoverable displacement vectors.
2. Test whether those directions compose nontrivially.
3. Separate semantic order effects from wording artifacts.
4. Test local third-order signed permutation structure with null baselines.
5. Expand from hand-written probes to systematic grammar-generated probes.
6. Decide which claims are strong enough for publication.

## ## Common methodology standards

Every promoted claim should have at least one control:

- holdout split
- source-only and target-only baseline
- permutation/null baseline
- semantic equivalence control
- bootstrap confidence interval
- cross-model replication
- template de-duplication check

Negative results remain part of the research record.

## ## Next research plan

### ### Short term

1. Close methodology blockers required for any submission:
  - remove antisymmetry from the evidence narrative; already done in the draft, keep it that way
  - add Procrustes null baselines
  - increase shuffle/permutation controls to at least 1000, ideally 5000 for final numbers
  - add commutator norm null baselines
2. Resolve UPAT:
  - either expand UPAT and match train sizes against the main dataset
  - or keep it explicitly as a hard negative control and narrow Track 1 claims
3. Add null baselines for exploratory UPAT alignment:
  - random-label or random-pairing Procrustes null
  - at least 1000 shuffle permutations instead of 100
4. Test cross-model transformation transfer as the likely central future paper:
  - reverse-direction transfer
  - alignment-size curve

- random-label/null alignment controls

5. Treat the syntax holdout as resolved for the current draft: `y\_only=1.0` and layer-0 `1.0` mean the `syntax=1.0` result is a target/surface artifact unless a future redesigned split proves otherwise.
6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.
7. Build a grammar-driven template generator for `N,Q,M,T` that produces many paraphrases without duplicate endpoints.
8. Add automated dataset validation:
  - no duplicate endpoint strings within a composition tuple
  - balanced subjects/actions
  - controlled lexical overlap
9. Add commutator null baselines for `|[A,B]|` using random or label-shuffled operations with matched norms.
10. Re-run composition and signed-permutation diagnostics on generated templates.
11. Add target-only and endpoint-only baselines for the Lie-style paper.
12. Regenerate a dated PDF packet after every major run.

### ### Medium term

1. Run the GPT-2 steering-vector experiment for one transformation, starting with negation or question formation.
2. Run the cross-lingual mBERT/XLM-R transformation-transfer experiment.
3. Measure effective dimensionality of transformation subspaces:
  - participation ratio
  - PCA retention curves
  - accuracy versus retained dimensions
4. Run layerwise and pooling ablations before expanding the operator set. If final-layer mean pooling is not the best representation, the expanded operator experiments should use the validated representation instead.
5. Add more linguistic operators:
  - passive voice
  - modality strength
  - evidentiality
  - aspect
  - conditionality
  - quantifier changes
6. Add paraphrase robustness with semantically equivalent endpoint variants.

### ### Paper milestones

1. Track 1 is arXiv-ready only when this checklist is complete:
  - syntax holdout breakdown is in the draft and interpreted as endpoint leakage
  - McNemar tests are reported in the text

- multiseed standard deviations are reported
- at least one prior-work baseline or comparison is written up, such as task vectors or function

vectors

- at least one model outside the original five is included as a spot-check; currently satisfied

by BERT-large and DeBERTa-v3-base

- UPAT hard-holdout is either resolved experimentally or explicitly reported as a negative

boundary condition

2. Keep Track 2 as a diagnostics paper until grammar-generated templates and endpoint-only controls succeed.

3. Promote Track 3 to the main paper only if Procrustes transfer survives null baselines. If it does, the main narrative becomes cross-architecture universal transformation subspaces.

4. If steering works, write Track 4 as an intervention/controllable-generation paper.

5. If cross-lingual transfer works, it becomes the strongest version of the universality claim.

6. If Track 2 survives grammar-generated controls, write it as a separate diagnostics paper rather than merging it into Track 1.

7. If Track 2 weakens under controls, keep it as a negative/diagnostic section in a broader research note.

## Code listing: run\_lie\_algebraic\_identities.py (1)

```
0001: import gc
0002: import json
0003: import os
0004: import warnings
0005: from pathlib import Path
0006:
0007: import numpy as np
0008: import pandas as pd
0009: import torch
0010: import matplotlib.pyplot as plt
0011:
0012: from transformers import AutoTokenizer, AutoModel
0013: from sklearn.decomposition import PCA
0014: from sklearn.metrics.pairwise import cosine_similarity
0015:
0016: warnings.filterwarnings("ignore")
0017:
0018:
0019: class LieAlgebraicIdentitiesAudit:
0020:     def __init__(self):
0021:         self.out_dir = Path(os.getenv("LIE_ALGEBRA_OUT_DIR", "lie_algebraic_identities_results"))
0022:         self.csv_dir = self.out_dir / "csv"
0023:         self.fig_dir = self.out_dir / "figures"
0024:         self.csv_dir.mkdir(parents=True, exist_ok=True)
0025:         self.fig_dir.mkdir(parents=True, exist_ok=True)
0026:
0027:         self.device = "cuda" if torch.cuda.is_available() else "cpu"
0028:         self.hf_token = None
0029:         self.pca_dim = 64
0030:         self.n_jacobi_null = 1000
0031:         self.n_bootstrap = 2000
0032:
0033:         self.models = [
0034:             model.strip()
0035:             for model in os.getenv(
0036:                 "LIE_ALGEBRA_MODELS",
0037:                 "bert-base-uncased,distilroberta-base,roberta-base",
0038:             ).split(",")
0039:             if model.strip()
0040:         ]
0041:
0042:         self.ops = ["N", "Q", "M", "T"]
0043:
0044:         self.subjects = [
0045:             "scientist", "engineer", "teacher", "doctor", "programmer",
0046:             "researcher", "analyst", "manager", "wizard", "dragon",
0047:             "queen", "robot", "pirate", "oracle", "knight", "alien",
0048:         ]
0049:
0050:         self.actions = [
0051:             ("accepted", "accept", "the explanation"),
0052:             ("completed", "complete", "the repair"),
0053:             ("confirmed", "confirm", "the answer"),
0054:             ("approved", "approve", "the treatment"),
```

## Code listing: run\_lie\_algebraic\_identities.py (2)

```
0055:         ("fixed", "fix", "the bug"),
0056:         ("supported", "support", "the theory"),
0057:         ("verified", "verify", "the report"),
0058:         ("guarded", "guard", "the treasure"),
0059:         ("opened", "open", "the portal"),
0060:         ("signed", "sign", "the treaty"),
0061:     ]
0062:
0063:     def log(self, msg):
0064:         print(msg, flush=True)
0065:
0066:     def base_sentence(self, subject, past, obj):
0067:         return f"The {subject} {past} {obj}."
0068:
0069:     def compose(self, seq, subject, past, base, obj):
0070:         s = "".join(seq)
0071:
0072:         mapping = {
0073:             "": f"The {subject} {past} {obj}.",
0074:
0075:             "N": f"The {subject} failed to {base} {obj}.",
0076:             "Q": f"Could the {subject} {base} {obj}?",
0077:             "M": f"The {subject} allegedly {past} {obj}.",
0078:             "T": f"The {subject} will {base} {obj} tomorrow.",
0079:
0080:             "NQ": f"Could the {subject} fail to {base} {obj}?",
0081:             "QN": f"Is it false that the {subject} {past} {obj}?",
0082:
0083:             "NM": f"The {subject} allegedly failed to {base} {obj}.",
0084:             "MN": f"Allegedly, the {subject} failed to {base} {obj}.",
0085:
0086:             "NT": f"The {subject} will fail to {base} {obj} tomorrow.",
0087:             "TN": f"The {subject} failed to have {past} {obj} earlier.",
0088:
0089:             "QM": f"Could the {subject} allegedly {base} {obj}?",
0090:             "MQ": f"Is it alleged that the {subject} {past} {obj}?",
0091:
0092:             "QT": f"Could the {subject} {base} {obj} tomorrow?",
0093:             "TQ": f"Will the {subject} {base} {obj} tomorrow?",
0094:
0095:             "MT": f"The {subject} will allegedly {base} {obj} tomorrow.",
0096:             "TM": f"The {subject} was allegedly going to {base} {obj}.",
0097:         }
0098:
0099:         if s in mapping:
0100:             return mapping[s]
0101:
0102:         # canonical triple templates
0103:         if s == "NQM":
0104:             return f"Could the {subject} allegedly fail to {base} {obj}?"
0105:         if s == "QMN":
0106:             return f"Is it false that the {subject} allegedly {past} {obj}?"
0107:         if s == "MNQ":
0108:             return f"Is it alleged that the {subject} failed to {base} {obj}?"
```

## Code listing: run\_lie\_algebraic\_identities.py (3)

```
0109:
0110:     if s == "QNM":
0111:         return f"Could it be false that the {subject} allegedly {past} {obj}?"
0112:     if s == "NMQ":
0113:         return f"Could it be alleged that the {subject} failed to {base} {obj}?"
0114:     if s == "MQN":
0115:         return f"Allegedly, is it false that the {subject} {past} {obj}?"
0116:
0117:     if s == "NMT":
0118:         return f"The {subject} will allegedly fail to {base} {obj} tomorrow."
0119:     if s == "MTN":
0120:         return f"The {subject} was allegedly going to fail to {base} {obj}."
0121:     if s == "TNM":
0122:         return f"Earlier, the {subject} allegedly failed to have {past} {obj}."
0123:
0124:     if s == "NTM":
0125:         return f"Allegedly, the {subject} will fail to {base} {obj} tomorrow."
0126:     if s == "TMN":
0127:         return f"Earlier, it was alleged that the {subject} failed to have {past} {obj}."
0128:     if s == "MNT":
0129:         return f"It is alleged that the {subject} will fail to {base} {obj} tomorrow."
0130:
0131:     if s == "NQT":
0132:         return f"Could the {subject} fail to {base} {obj} tomorrow?"
0133:     if s == "QTN":
0134:         return f"Is it false that the {subject} will {base} {obj} tomorrow?"
0135:     if s == "TNQ":
0136:         return f"Could the {subject} have failed to {base} {obj} earlier?"
0137:
0138:     if s == "QNT":
0139:         return f"Could it be false that the {subject} will {base} {obj} tomorrow?"
0140:     if s == "NTQ":
0141:         return f"Could it be that the {subject} will fail to {base} {obj} tomorrow?"
0142:     if s == "TQN":
0143:         return f"Will it be false that the {subject} {base} {obj} tomorrow?"
0144:
0145:     if s == "QMT":
0146:         return f"Could the {subject} allegedly {base} {obj} tomorrow?"
0147:     if s == "MTQ":
0148:         return f"Could it be alleged that the {subject} will {base} {obj} tomorrow?"
0149:     if s == "TQM":
0150:         return f"Allegedly, will the {subject} {base} {obj} tomorrow?"
0151:
0152:     if s == "MQT":
0153:         return f"Will it be alleged that the {subject} {base} {obj} tomorrow?"
0154:     if s == "QTM":
0155:         return f"Allegedly, could the {subject} {base} {obj} tomorrow?"
0156:     if s == "TMQ":
0157:         return f"Could the {subject} have allegedly planned to {base} {obj}?"
0158:
0159:     raise ValueError(f"Unsupported composition: {s}")
0160:
0161: def build_dataset(self, n_templates=100, seed=42):
0162:     rng = np.random.default_rng(seed)
```

## Code listing: run\_lie\_algebraic\_identities.py (4)

```
0163:         combos = [(s, a) for s in self.subjects for a in self.actions]
0164:         rng.shuffle(combos)
0165:         combos = combos[:n_templates]
0166:
0167:         rows = []
0168:
0169:         pairs = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
0170:         triples = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]
0171:
0172:         for i, (subject, action) in enumerate(combos):
0173:             past, base, obj = action
0174:             source = self.base_sentence(subject, past, obj)
0175:
0176:             for a, b in pairs:
0177:                 rows.append({
0178:                     "template_id": i,
0179:                     "kind": "antisymmetry",
0180:                     "source": source,
0181:                     "a": a,
0182:                     "b": b,
0183:                     "c": "",
0184:                     "ab": self.compose([a, b], subject, past, base, obj),
0185:                     "ba": self.compose([b, a], subject, past, base, obj),
0186:                 })
0187:
0188:             for a, b, c in triples:
0189:                 rows.append({
0190:                     "template_id": i,
0191:                     "kind": "jacobi",
0192:                     "source": source,
0193:                     "a": a,
0194:                     "b": b,
0195:                     "c": c,
0196:                     "abc": self.compose([a, b, c], subject, past, base, obj),
0197:                     "bca": self.compose([b, c, a], subject, past, base, obj),
0198:                     "cab": self.compose([c, a, b], subject, past, base, obj),
0199:                     "acb": self.compose([a, c, b], subject, past, base, obj),
0200:                     "cba": self.compose([c, b, a], subject, past, base, obj),
0201:                     "bac": self.compose([b, a, c], subject, past, base, obj),
0202:                 })
0203:
0204:         df = pd.DataFrame(rows)
0205:         df.to_csv(self.csv_dir / "lie_algebraic_identities_dataset.csv", index=False)
0206:         return df
0207:
0208:     @torch.no_grad()
0209:     def get_embeddings(self, model_name, texts, batch_size=16):
0210:         tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)
0211:
0212:         if tokenizer.pad_token is None:
0213:             tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0214:
0215:         model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
0216:         model.eval()
```

## Code listing: run\_lie\_algebraic\_identities.py (5)

```
0217:
0218:         embs = []
0219:
0220:         for i in range(0, len(texts), batch_size):
0221:             batch = texts[i:i + batch_size]
0222:
0223:             enc = tokenizer(
0224:                 batch,
0225:                 padding=True,
0226:                 truncation=True,
0227:                 max_length=128,
0228:                 return_tensors="pt",
0229:             ).to(self.device)
0230:
0231:             out = model(**enc)
0232:             hidden = out.last_hidden_state
0233:             mask = enc["attention_mask"].unsqueeze(-1)
0234:
0235:             pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
0236:             embs.append(pooled.detach().cpu().numpy())
0237:
0238:         del model
0239:         gc.collect()
0240:
0241:         if self.device == "cuda":
0242:             torch.cuda.empty_cache()
0243:
0244:         return np.vstack(embs)
0245:
0246:     def cos(self, x, y):
0247:         return float(cosine_similarity(x.reshape(1, -1), y.reshape(1, -1))[0, 0])
0248:
0249:     def alternating_null_stats(self, vectors, observed_norm, scale, rng):
0250:         null_relative = []
0251:         n = len(vectors)
0252:
0253:         for _ in range(self.n_jacobi_null):
0254:             positive = set(rng.choice(n, size=n // 2, replace=False).tolist())
0255:             signed_sum = np.zeros_like(vectors[0])
0256:
0257:             for i, vector in enumerate(vectors):
0258:                 signed_sum += vector if i in positive else -vector
0259:
0260:             null_relative.append(np.linalg.norm(signed_sum) / scale)
0261:
0262:         null_relative = np.asarray(null_relative, dtype=float)
0263:         observed_relative = observed_norm / scale
0264:
0265:         return {
0266:             "null_mean_relative_jacobi_norm": float(null_relative.mean()),
0267:             "null_std_relative_jacobi_norm": float(null_relative.std(ddof=1)),
0268:             "jacobi_to_null_mean_ratio": float(observed_relative / (null_relative.mean() + 1e-12)),
0269:             "null_percentile_smaller_is_better": float((null_relative <= observed_relative).mean()),
0270:         }
```

## Code listing: run\_lie\_algebraic\_identities.py (6)

```
0271:
0272:     def bootstrap_mean_ci(self, values, rng, alpha=0.05):
0273:         values = np.asarray(values, dtype=float)
0274:         if len(values) == 0:
0275:             return np.nan, np.nan
0276:
0277:         draws = rng.choice(values, size=(self.n_bootstrap, len(values)), replace=True).mean(axis=1)
0278:         return (
0279:             float(np.quantile(draws, alpha / 2)),
0280:             float(np.quantile(draws, 1 - alpha / 2)),
0281:         )
0282:
0283:     def compute_for_model(self, model_name, df):
0284:         self.log("\n" + "=" * 80)
0285:         self.log(f"MODEL: {model_name}")
0286:         self.log("=" * 80)
0287:
0288:         text_cols = ["source", "ab", "ba", "abc", "bca", "cab", "acb", "cba", "bac"]
0289:         all_texts = set()
0290:
0291:         for col in text_cols:
0292:             if col in df.columns:
0293:                 all_texts |= set(df[col].dropna().values)
0294:
0295:         all_texts = sorted(all_texts)
0296:         idx = {t: i for i, t in enumerate(all_texts)}
0297:
0298:         raw = self.get_embeddings(model_name, all_texts)
0299:
0300:         dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
0301:         pca = PCA(n_components=dim, random_state=42)
0302:         vecs = pca.fit_transform(raw)
0303:
0304:         anti_rows = []
0305:         jacobi_rows = []
0306:
0307:         anti_df = df[df["kind"] == "antisymmetry"]
0308:
0309:         for row in anti_df.itertuples():
0310:             x = vecs[idx[row.source]]
0311:             ab = vecs[idx[row.ab]]
0312:             ba = vecs[idx[row.ba]]
0313:
0314:             delta_ab = ab - x
0315:             delta_ba = ba - x
0316:
0317:             comm_ab = delta_ab - delta_ba
0318:             comm_ba = delta_ba - delta_ab
0319:
0320:             antisym_error = np.linalg.norm(comm_ab + comm_ba)
0321:             comm_norm = np.linalg.norm(comm_ab)
0322:
0323:             anti_rows.append({
0324:                 "model": model_name,
```

## Code listing: run\_lie\_algebraic\_identities.py (7)

```
0325:         "template_id": row.template_id,
0326:         "pair": f"{row.a}{row.b}_vs_{row.b}{row.a}",
0327:         "a": row.a,
0328:         "b": row.b,
0329:         "comm_norm": float(comm_norm),
0330:         "antisym_error": float(antisym_error),
0331:         "relative_antisym_error": float(antisym_error / (comm_norm + 1e-12)),
0332:         "cos_comm_ab_neg_comm_ba": self.cos(comm_ab, -comm_ba),
0333:     })
0334:
0335: jacobi_df = df[df["kind"] == "jacobi"]
0336: seed = sum((i + 1) * ord(ch) for i, ch in enumerate(model_name)) % (2 ** 32)
0337: rng = np.random.default_rng(seed)
0338:
0339: for row in jacobi_df.itertuples():
0340:     x = vecs[idx[row.source]]
0341:
0342:     abc = vecs[idx[row.abc]] - x
0343:     bca = vecs[idx[row.bca]] - x
0344:     cab = vecs[idx[row.cab]] - x
0345:
0346:     acb = vecs[idx[row.acb]] - x
0347:     cba = vecs[idx[row.cba]] - x
0348:     bac = vecs[idx[row.bac]] - x
0349:
0350:     # Jacobi-like alternating sum over permutations:
0351:     # J = ABC + BCA + CAB - ACB - CBA - BAC
0352:     jacobi = abc + bca + cab - acb - cba - bac
0353:
0354:     positive_norm = np.linalg.norm(abc) + np.linalg.norm(bca) + np.linalg.norm(cab)
0355:     negative_norm = np.linalg.norm(acb) + np.linalg.norm(cba) + np.linalg.norm(bac)
0356:     scale = 0.5 * (positive_norm + negative_norm) + 1e-12
0357:
0358:     jacobi_norm = np.linalg.norm(jacobi)
0359:     null_stats = self.alternating_null_stats(
0360:         [abc, bca, cab, acb, cba, bac],
0361:         jacobi_norm,
0362:         scale,
0363:         rng,
0364:     )
0365:
0366:     jacobi_rows.append({
0367:         "model": model_name,
0368:         "template_id": row.template_id,
0369:         "triple": f"{row.a}{row.b}{row.c}",
0370:         "a": row.a,
0371:         "b": row.b,
0372:         "c": row.c,
0373:         "jacobi_norm": float(jacobi_norm),
0374:         "relative_jacobi_norm": float(jacobi_norm / scale),
0375:         **null_stats,
0376:     })
0377:
0378: anti_result = pd.DataFrame(anti_rows)
```

## Code listing: run\_lie\_algebraic\_identities.py (8)

```
0379:         jacobi_result = pd.DataFrame(jacobi_rows)
0380:
0381:         anti_result.to_csv(self.csv_dir / f"antisymmetry_raw_{self.safe_name(model_name)}.csv", index=False)
0382:         jacobi_result.to_csv(self.csv_dir / f"jacobi_raw_{self.safe_name(model_name)}.csv", index=False)
0383:
0384:         return anti_result, jacobi_result
0385:
0386:     def safe_name(self, model_name):
0387:         return model_name.replace("/", "_").replace("-", "_")
0388:
0389:     def summarize(self, anti_all, jacobi_all):
0390:         anti_summary = (
0391:             anti_all.groupby(["model", "pair"])
0392:             .agg(
0393:                 mean_comm_norm=("comm_norm", "mean"),
0394:                 mean_relative_antisym_error=("relative_antisym_error", "mean"),
0395:                 mean_cos_comm_ab_neg_comm_ba=("cos_comm_ab_neg_comm_ba", "mean"),
0396:                 n=("pair", "count"),
0397:             )
0398:             .reset_index()
0399:         )
0400:
0401:         jacobi_summary = (
0402:             jacobi_all.groupby(["model", "triple"])
0403:             .agg(
0404:                 mean_jacobi_norm=("jacobi_norm", "mean"),
0405:                 mean_relative_jacobi_norm=("relative_jacobi_norm", "mean"),
0406:                 mean_null_relative_jacobi_norm=("null_mean_relative_jacobi_norm", "mean"),
0407:                 mean_jacobi_to_null_mean_ratio=("jacobi_to_null_mean_ratio", "mean"),
0408:                 mean_null_percentile_smaller_is_better=("null_percentile_smaller_is_better", "mean"),
0409:                 n=("triple", "count"),
0410:             )
0411:             .reset_index()
0412:         )
0413:
0414:         rng = np.random.default_rng(20260611)
0415:         ci_rows = []
0416:
0417:         for row in jacobi_summary.itertuples():
0418:             group = jacobi_all[
0419:                 (jacobi_all["model"] == row.model)
0420:                 & (jacobi_all["triple"] == row.triple)
0421:             ]
0422:
0423:             rel_low, rel_high = self.bootstrap_mean_ci(group["relative_jacobi_norm"].values, rng)
0424:             ratio_low, ratio_high = self.bootstrap_mean_ci(group["jacobi_to_null_mean_ratio"].values, rng)
0425:
0426:             ci_rows.append({
0427:                 "model": row.model,
0428:                 "triple": row.triple,
0429:                 "relative_jacobi_norm_ci95_low": rel_low,
0430:                 "relative_jacobi_norm_ci95_high": rel_high,
0431:                 "jacobi_to_null_mean_ratio_ci95_low": ratio_low,
0432:                 "jacobi_to_null_mean_ratio_ci95_high": ratio_high,
```

## Code listing: run\_lie\_algebraic\_identities.py (9)

```
0433:         })
0434:
0435:         jacobi_summary = jacobi_summary.merge(pd.DataFrame(ci_rows), on=["model", "triple"], how="left")
0436:
0437:         anti_summary.to_csv(self.csv_dir / "antisymmetry_summary.csv", index=False)
0438:         jacobi_summary.to_csv(self.csv_dir / "jacobi_summary.csv", index=False)
0439:
0440:         return anti_summary, jacobi_summary
0441:
0442:     def plot_heatmap(self, df, index_col, value_col, filename, title):
0443:         rows = sorted(df["model"].unique())
0444:         cols = sorted(df[index_col].unique())
0445:
0446:         mat = np.zeros((len(rows), len(cols)))
0447:
0448:         for i, model in enumerate(rows):
0449:             for j, col in enumerate(cols):
0450:                 val = df[(df["model"] == model) & (df[index_col] == col)][value_col].values
0451:                 mat[i, j] = val[0] if len(val) else np.nan
0452:
0453:         plt.figure(figsize=(10, 5))
0454:         plt.imshow(mat, aspect="auto")
0455:         plt.colorbar(label=value_col)
0456:
0457:         plt.xticks(np.arange(len(cols)), cols, rotation=35, ha="right")
0458:         plt.yticks(np.arange(len(rows)), rows)
0459:         plt.title(title)
0460:
0461:         for i in range(mat.shape[0]):
0462:             for j in range(mat.shape[1]):
0463:                 plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")
0464:
0465:         path = self.fig_dir / filename
0466:         plt.tight_layout()
0467:         plt.savefig(path, dpi=220, bbox_inches="tight")
0468:         plt.close()
0469:         self.log(f"Saved figure: {path}")
0470:
0471:     def run(self):
0472:         self.log(f"DEVICE: {self.device}")
0473:         self.log(f"OUT DIR: {self.out_dir}")
0474:
0475:         df = self.build_dataset(n_templates=100, seed=42)
0476:
0477:         anti_results = []
0478:         jacobi_results = []
0479:
0480:         for model in self.models:
0481:             anti, jacobi = self.compute_for_model(model, df)
0482:             anti_results.append(anti)
0483:             jacobi_results.append(jacobi)
0484:
0485:         anti_all = pd.concat(anti_results, ignore_index=True)
0486:         jacobi_all = pd.concat(jacobi_results, ignore_index=True)
```

## Code listing: run\_lie\_algebraic\_identities.py (10)

```
0487:
0488:     anti_all.to_csv(self.csv_dir / "antisymmetry_raw_all_models.csv", index=False)
0489:     jacobi_all.to_csv(self.csv_dir / "jacobi_raw_all_models.csv", index=False)
0490:
0491:     anti_summary, jacobi_summary = self.summarize(anti_all, jacobi_all)
0492:
0493:     self.plot_heatmap(
0494:         anti_summary,
0495:         "pair",
0496:         "mean_cos_comm_ab_neg_comm_ba",
0497:         "01_antisymmetry_cosine_heatmap.png",
0498:         "Antisymmetry Check:  $\cos([A,B], -[B,A])$ ",
0499:     )
0500:
0501:     self.plot_heatmap(
0502:         anti_summary,
0503:         "pair",
0504:         "mean_relative_antisym_error",
0505:         "02_antisymmetry_error_heatmap.png",
0506:         "Antisymmetry Error",
0507:     )
0508:
0509:     self.plot_heatmap(
0510:         jacobi_summary,
0511:         "triple",
0512:         "mean_relative_jacobi_norm",
0513:         "03_jacobi_relative_norm_heatmap.png",
0514:         "Jacobi-like Alternating Sum Relative Norm",
0515:     )
0516:
0517:     self.plot_heatmap(
0518:         jacobi_summary,
0519:         "triple",
0520:         "mean_jacobi_to_null_mean_ratio",
0521:         "04_jacobi_vs_permutation_null_heatmap.png",
0522:         "Jacobi-like Norm / Permutation Null Mean",
0523:     )
0524:
0525:     metadata = {
0526:         "antisymmetry": "checks whether  $[A,B] \approx -[B,A]$ ",
0527:         "jacobi_like": "checks alternating sum  $ABC+BCA+CAB-ACB-CBA-BAC$ ",
0528:         "jacobi_null": (
0529:             "compares the observed alternating sum with random 3-positive/3-negative "
0530:             "sign assignments over the same six composition vectors; lower ratio and "
0531:             "lower percentile mean stronger Jacobi-like cancellation"
0532:         ),
0533:         "caution": (
0534:             "A literal nested-commutator Jacobi expansion over the same six embedded "
0535:             "composition endpoints cancels by construction, so this script reports a "
0536:             "non-tautological alternating third-order cancellation diagnostic instead."
0537:         ),
0538:         "note": "This is an empirical Lie-style diagnostic, not a formal proof of Lie algebra.",
0539:         "models": self.models,
0540:         "n_jacobi_null": self.n_jacobi_null,
```

## Code listing: run\_lie\_algebraic\_identities.py (11)

```
0541:         "n_bootstrap": self.n_bootstrap,
0542:         "jacobi_triples": ["NQM", "NQT", "NMT", "QMT"],
0543:     }
0544:
0545:     with open(self.out_dir / "metadata.json", "w", encoding="utf-8") as f:
0546:         json.dump(metadata, f, indent=2)
0547:
0548:     self.log("\nANTISYMMETRY SUMMARY")
0549:     self.log(str(anti_summary))
0550:
0551:     self.log("\nJACOBI SUMMARY")
0552:     self.log(str(jacobi_summary))
0553:
0554:     self.log("\nDONE")
0555:
0556:
0557: if __name__ == "__main__":
0558:     audit = LieAlgebraicIdentitiesAudit()
0559:     audit.run()
```

## Code listing: run\_syntax\_representation\_ablation.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007:
0008: from sklearn.linear_model import LogisticRegression
0009: from sklearn.metrics import accuracy_score
0010: from sklearn.pipeline import make_pipeline
0011: from sklearn.preprocessing import StandardScaler
0012: from sklearn.svm import LinearSVC
0013:
0014: import lie_llm_syntax_holdout_experiment as syntax
0015:
0016:
0017: OUT_DIR = Path(os.getenv("SYNTAX_ABLATION_OUT_DIR", "syntax_representation_ablation_results"))
0018: OUT_DIR.mkdir(parents=True, exist_ok=True)
0019:
0020: MODELS = [m.strip() for m in os.getenv(
0021:     "SYNTAX_ABLATION_MODELS",
0022:     "bert-base-uncased,roberta-base,distilroberta-base,gpt2,distilgpt2",
0023: ).split(",") if m.strip()]
0024:
0025: N_BASE = int(os.getenv("SYNTAX_ABLATION_N_BASE", str(syntax.N_BASE)))
0026:
0027:
0028: def run_classifiers(X, labels, train_mask, test_mask):
0029:     classifiers = {
0030:         "logreg": make_pipeline(
0031:             StandardScaler(),
0032:             LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1),
0033:         ),
0034:         "linear_svc": make_pipeline(
0035:             StandardScaler(),
0036:             LinearSVC(class_weight="balanced", max_iter=20000),
0037:         ),
0038:     }
0039:
0040:     rows = []
0041:     for clf_name, clf in classifiers.items():
0042:         clf.fit(X[train_mask], labels[train_mask])
0043:         pred = clf.predict(X[test_mask])
0044:         rows.append({
0045:             "classifier": clf_name,
0046:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0047:             "n_train": int(train_mask.sum()),
0048:             "n_test": int(test_mask.sum()),
0049:         })
0050:
0051:     return rows
0052:
0053:
0054: def main():
```

## Code listing: run\_syntax\_representation\_ablation.py (2)

```
0055:     syntax.OUT_DIR = str(OUT_DIR)
0056:     os.makedirs(syntax.OUT_DIR, exist_ok=True)
0057:
0058:     base_df = syntax.make_base_rows(N_BASE)
0059:     pair_df = syntax.build_pairs(base_df)
0060:     prompts, pair_df = syntax.index_prompts(pair_df)
0061:
0062:     base_df.to_csv(OUT_DIR / "base_sentences.csv", index=False)
0063:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0064:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(
0065:         OUT_DIR / "prompts.csv",
0066:         index=False,
0067:     )
0068:
0069:     labels = pair_df["class"].astype(str).to_numpy()
0070:     syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
0071:     train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
0072:     test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
0073:     src_idx = pair_df["source_idx"].to_numpy()
0074:     tgt_idx = pair_df["target_idx"].to_numpy()
0075:
0076:     all_rows = []
0077:
0078:     for model_name in MODELS:
0079:         print(f"\n=== MODEL: {model_name} ===")
0080:         vectors = syntax.get_vectors(model_name, prompts)
0081:
0082:         x_src = vectors[src_idx]
0083:         x_tgt = vectors[tgt_idx]
0084:         representations = {
0085:             "x_only": x_src,
0086:             "y_only": x_tgt,
0087:             "delta": x_tgt - x_src,
0088:             "concat": np.concatenate([x_src, x_tgt], axis=1),
0089:         }
0090:
0091:         for rep_name, X in representations.items():
0092:             for row in run_classifiers(X, labels, train_mask, test_mask):
0093:                 row.update({
0094:                     "model": model_name,
0095:                     "representation": rep_name,
0096:                     "random_baseline": 1 / len(set(labels)),
0097:                     "n_base": N_BASE,
0098:                 })
0099:                 print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
0100:                 all_rows.append(row)
0101:
0102:         del vectors, x_src, x_tgt, representations
0103:         gc.collect()
0104:
0105:     result = pd.DataFrame(all_rows)
0106:     result.to_csv(OUT_DIR / "syntax_representation_ablation.csv", index=False)
0107:
0108:     pivot = result.pivot_table(
```

## Code listing: run\_syntax\_representation\_ablation.py (3)

```
0109:         index=["model", "classifier"],
0110:         columns="representation",
0111:         values="accuracy",
0112:     ).reset_index()
0113:     pivot.to_csv(OUT_DIR / "syntax_representation_ablation_pivot.csv", index=False)
0114:
0115:     print("\nSUMMARY")
0116:     print(pivot)
0117:     print("Saved to", OUT_DIR)
0118:
0119:
0120: if __name__ == "__main__":
0121:     main()
```

## Code listing: run\_layerwise\_pooling\_ablation.py (1)

```
0001: import gc
0002: import os
0003: import time
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: from sklearn.linear_model import LogisticRegression
0010: from sklearn.metrics import accuracy_score
0011: from sklearn.pipeline import make_pipeline
0012: from sklearn.preprocessing import StandardScaler
0013: from sklearn.svm import LinearSVC
0014: from transformers import AutoModel, AutoTokenizer
0015:
0016: import lie_llm_syntax_holdout_experiment as syntax
0017:
0018:
0019: OUT_DIR = Path(os.getenv("LAYERWISE_OUT_DIR", "layerwise_pooling_ablation_results"))
0020: OUT_DIR.mkdir(parents=True, exist_ok=True)
0021:
0022: MODEL_NAME = os.getenv("LAYERWISE_MODEL", "bert-base-uncased")
0023: N_BASE = int(os.getenv("LAYERWISE_N_BASE", "300"))
0024: BATCH_SIZE = int(os.getenv("LAYERWISE_BATCH_SIZE", "16"))
0025:
0026:
0027: def pool(hidden, mask, mode):
0028:     if mode == "mean":
0029:         m = mask.unsqueeze(-1).float()
0030:         return (hidden * m).sum(dim=1) / m.sum(dim=1).clamp_min(1e-9)
0031:     if mode == "cls":
0032:         return hidden[:, 0, :]
0033:     if mode == "last_token":
0034:         idx = mask.sum(dim=1).clamp_min(1) - 1
0035:         return hidden[torch.arange(hidden.shape[0]), idx]
0036:     raise ValueError(mode)
0037:
0038:
0039: def extract_layer_vectors(prompts):
0040:     tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
0041:     if tokenizer.pad_token is None:
0042:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0043:
0044:     model = AutoModel.from_pretrained(MODEL_NAME, output_hidden_states=True)
0045:     model.eval()
0046:
0047:     rows_by_key = {}
0048:     started = time.time()
0049:
0050:     for start in range(0, len(prompts), BATCH_SIZE):
0051:         batch = prompts[start:start + BATCH_SIZE]
0052:         enc = tokenizer(batch, padding=True, truncation=True, max_length=160, return_tensors="pt")
0053:
0054:         with torch.no_grad():
```

## Code listing: run\_layerwise\_pooling\_ablation.py (2)

```
0055:         out = model(**enc)
0056:
0057:         for layer_idx, hidden in enumerate(out.hidden_states):
0058:             for pooling in ["mean", "cls", "last_token"]:
0059:                 key = (layer_idx, pooling)
0060:                 rows_by_key.setdefault(key, []).append(pool(hidden, enc["attention_mask"], pooling).cpu().numpy())
0061:
0062:         done = start + len(batch)
0063:         print(f"{MODEL_NAME}: {done}/{len(prompts)}, {time.time() - started:.1f}s")
0064:
0065:     result = {key: np.vstack(parts) for key, parts in rows_by_key.items()}
0066:
0067:     del model
0068:     gc.collect()
0069:     return result
0070:
0071:
0072: def fit_score(X, labels, train_mask, test_mask):
0073:     classifiers = {
0074:         "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
0075:         "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
0076:     }
0077:
0078:     rows = []
0079:     for clf_name, clf in classifiers.items():
0080:         clf.fit(X[train_mask], labels[train_mask])
0081:         pred = clf.predict(X[test_mask])
0082:         rows.append({
0083:             "classifier": clf_name,
0084:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0085:         })
0086:     return rows
0087:
0088:
0089: def main():
0090:     base_df = syntax.make_base_rows(N_BASE)
0091:     pair_df = syntax.build_pairs(base_df)
0092:     prompts, pair_df = syntax.index_prompts(pair_df)
0093:
0094:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0095:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0096:
0097:     labels = pair_df["class"].astype(str).to_numpy()
0098:     syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
0099:     train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
0100:     test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
0101:     src_idx = pair_df["source_idx"].to_numpy()
0102:     tgt_idx = pair_df["target_idx"].to_numpy()
0103:
0104:     vectors_by_key = extract_layer_vectors(prompts)
0105:
0106:     all_rows = []
0107:     for (layer_idx, pooling), vectors in vectors_by_key.items():
0108:         x_src = vectors[src_idx]
```

## Code listing: run\_layerwise\_pooling\_ablation.py (3)

```
0109:         x_tgt = vectors[tgt_idx]
0110:
0111:     for rep_name, X in {
0112:         "delta": x_tgt - x_src,
0113:         "y_only": x_tgt,
0114:     }.items():
0115:         for row in fit_score(X, labels, train_mask, test_mask):
0116:             row.update({
0117:                 "model": MODEL_NAME,
0118:                 "layer": layer_idx,
0119:                 "pooling": pooling,
0120:                 "representation": rep_name,
0121:                 "n_base": N_BASE,
0122:                 "n_train": int(train_mask.sum()),
0123:                 "n_test": int(test_mask.sum()),
0124:             })
0125:             print(row)
0126:             all_rows.append(row)
0127:
0128:     result = pd.DataFrame(all_rows)
0129:     result.to_csv(OUT_DIR / "layerwise_pooling_ablation.csv", index=False)
0130:
0131:     best = (
0132:         result[result["classifier"] == "linear_svc"]
0133:         .sort_values("accuracy", ascending=False)
0134:         .head(20)
0135:     )
0136:     best.to_csv(OUT_DIR / "layerwise_pooling_ablation_top20.csv", index=False)
0137:     print(best)
0138:     print("Saved to", OUT_DIR)
0139:
0140:
0141: if __name__ == "__main__":
0142:     main()
```

## Code listing: run\_track1\_spotcheck.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007: from sklearn.linear_model import LogisticRegression
0008: from sklearn.metrics import accuracy_score
0009: from sklearn.pipeline import make_pipeline
0010: from sklearn.preprocessing import StandardScaler
0011: from sklearn.svm import LinearSVC
0012:
0013: import lie_llm_full_semantic_holdout_experiment as full
0014:
0015:
0016: OUT_DIR = Path(os.getenv("SPOTCHECK_OUT_DIR", "track1_spotcheck_results"))
0017: OUT_DIR.mkdir(parents=True, exist_ok=True)
0018:
0019: MODELS = [m.strip() for m in os.getenv("SPOTCHECK_MODELS", "bert-large-uncased").split(",") if m.strip()]
0020: N_BASE = int(os.getenv("SPOTCHECK_N_BASE", "100"))
0021:
0022:
0023: def run_classifiers(X, labels, split):
0024:     train_mask = split == "train"
0025:     test_mask = split == "test"
0026:     classifiers = {
0027:         "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
0028:         "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
0029:     }
0030:     rows = []
0031:     for clf_name, clf in classifiers.items():
0032:         clf.fit(X[train_mask], labels[train_mask])
0033:         pred = clf.predict(X[test_mask])
0034:         rows.append({
0035:             "classifier": clf_name,
0036:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0037:             "n_train": int(train_mask.sum()),
0038:             "n_test": int(test_mask.sum()),
0039:         })
0040:     return rows
0041:
0042:
0043: def main():
0044:     full.OUT_DIR = str(OUT_DIR)
0045:     full.N_BASE = N_BASE
0046:     os.makedirs(full.OUT_DIR, exist_ok=True)
0047:
0048:     pair_df = full.build_dataset()
0049:     prompts, pair_df = full.index_prompts(pair_df)
0050:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0051:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0052:
0053:     labels = pair_df["class"].astype(str).to_numpy()
0054:     split = pair_df["split"].astype(str).to_numpy()
```

## Code listing: run\_track1\_spotcheck.py (2)

```
0055:     src_idx = pair_df["source_idx"].to_numpy()
0056:     tgt_idx = pair_df["target_idx"].to_numpy()
0057:
0058:     rows = []
0059:     for model_name in MODELS:
0060:         print(f"\n=== MODEL: {model_name} ===")
0061:         vectors = full.get_vectors(model_name, prompts)
0062:         x_src = vectors[src_idx]
0063:         x_tgt = vectors[tgt_idx]
0064:         reps = {
0065:             "x_only": x_src,
0066:             "y_only": x_tgt,
0067:             "delta": x_tgt - x_src,
0068:             "concat": np.concatenate([x_src, x_tgt], axis=1),
0069:         }
0070:         for rep_name, X in reps.items():
0071:             for row in run_classifiers(X, labels, split):
0072:                 row.update({
0073:                     "model": model_name,
0074:                     "representation": rep_name,
0075:                     "random_baseline": 1 / len(set(labels)),
0076:                     "n_base": N_BASE,
0077:                 })
0078:                 print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
0079:                 rows.append(row)
0080:     del vectors, reps
0081:     gc.collect()
0082:
0083:     result = pd.DataFrame(rows)
0084:     result.to_csv(OUT_DIR / "spotcheck_representation_ablation.csv", index=False)
0085:     pivot = result.pivot_table(index=["model", "classifier"], columns="representation", values="accuracy").reset_index()
0086:     pivot.to_csv(OUT_DIR / "spotcheck_representation_ablation_pivot.csv", index=False)
0087:     print(pivot)
0088:
0089:
0090: if __name__ == "__main__":
0091:     main()
```

## Code listing: run\_full\_semantic\_pooling\_ablation.py (1)

```
0001: import gc
0002: import os
0003: import random
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: from sklearn.linear_model import LogisticRegression
0010: from sklearn.metrics import accuracy_score
0011: from sklearn.pipeline import make_pipeline
0012: from sklearn.preprocessing import StandardScaler
0013: from sklearn.svm import LinearSVC
0014: from transformers import AutoModel, AutoTokenizer
0015:
0016: import lie_llm_full_semantic_holdout_experiment as full
0017:
0018:
0019: OUT_DIR = Path(os.getenv("POOLING_OUT_DIR", "full_semantic_pooling_ablation_results"))
0020: MODELS = [m.strip() for m in os.getenv("POOLING_MODELS", "bert-base-uncased,roberta-base,gpt2").split(",") if m.strip()]
0021: N_BASE = int(os.getenv("POOLING_N_BASE", "150"))
0022: BATCH_SIZE = int(os.getenv("POOLING_BATCH_SIZE", "32"))
0023: SEED = int(os.getenv("POOLING_SEED", "42"))
0024:
0025:
0026: def reset_full_dataset_seed():
0027:     full.N_BASE = N_BASE
0028:     full.random.seed(SEED)
0029:     full.np.random.seed(SEED)
0030:     random.seed(SEED)
0031:     np.random.seed(SEED)
0032:     torch.manual_seed(SEED)
0033:
0034:
0035: def safe_name(model_name):
0036:     return model_name.replace("/", "__")
0037:
0038:
0039: def last_token_pool(hidden, attention_mask):
0040:     lengths = attention_mask.sum(dim=1).clamp(min=1) - 1
0041:     batch_idx = torch.arange(hidden.shape[0], device=hidden.device)
0042:     return hidden[batch_idx, lengths]
0043:
0044:
0045: def get_pooled_vectors(model_name, prompts):
0046:     tokenizer = AutoTokenizer.from_pretrained(model_name)
0047:     if tokenizer.pad_token is None:
0048:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0049:
0050:     model = AutoModel.from_pretrained(model_name, output_hidden_states=True)
0051:     model.eval()
0052:
0053:     pooled = {"mean": [], "cls": [], "last_token": []}
0054:
```

## Code listing: run\_full\_semantic\_pooling\_ablation.py (2)

```
0055:     for start in range(0, len(prompts), BATCH_SIZE):
0056:         batch = prompts[start:start + BATCH_SIZE]
0057:         inputs = tokenizer(
0058:             batch,
0059:             padding=True,
0060:             truncation=True,
0061:             max_length=180,
0062:             return_tensors="pt",
0063:         )
0064:
0065:         with torch.no_grad():
0066:             out = model(**inputs)
0067:
0068:             hidden = out.hidden_states[-1]
0069:             mask = inputs["attention_mask"]
0070:
0071:             pooled["mean"].append(full.mean_pool(hidden, mask).cpu().numpy())
0072:             pooled["cls"].append(hidden[:, 0, :].cpu().numpy())
0073:             pooled["last_token"].append(last_token_pool(hidden, mask).cpu().numpy())
0074:
0075:             done = start + len(batch)
0076:             print(f"  {model_name}: {done}/{len(prompts)}", flush=True)
0077:
0078:     del model
0079:     gc.collect()
0080:
0081:     return {k: np.vstack(v) for k, v in pooled.items()}
0082:
0083:
0084: def build_features(vectors, pair_df, representation):
0085:     src = vectors[pair_df["source_idx"].to_numpy()]
0086:     tgt = vectors[pair_df["target_idx"].to_numpy()]
0087:
0088:     if representation == "x_only":
0089:         return src
0090:     if representation == "y_only":
0091:         return tgt
0092:     if representation == "delta":
0093:         return tgt - src
0094:     if representation == "concat":
0095:         return np.hstack([src, tgt])
0096:     raise ValueError(representation)
0097:
0098:
0099: def run_one(model_name, pooling, representation, vectors, pair_df):
0100:     X = build_features(vectors, pair_df, representation)
0101:     y = pair_df["class"].astype(str).to_numpy()
0102:     split = pair_df["split"].astype(str).to_numpy()
0103:
0104:     train = split == "train"
0105:     test = split == "test"
0106:
0107:     classifiers = {
0108:         "logreg": make_pipeline(
```

## Code listing: run\_full\_semantic\_pooling\_ablation.py (3)

```
0109:         StandardScaler(),
0110:         LogisticRegression(max_iter=5000, class_weight="balanced"),
0111:     ),
0112:     "linear_svc": make_pipeline(
0113:         StandardScaler(),
0114:         LinearSVC(class_weight="balanced", max_iter=20000),
0115:     ),
0116: }
0117:
0118: rows = []
0119: for clf_name, clf in classifiers.items():
0120:     clf.fit(X[train], y[train])
0121:     pred = clf.predict(X[test])
0122:     rows.append({
0123:         "model": model_name,
0124:         "pooling": pooling,
0125:         "representation": representation,
0126:         "classifier": clf_name,
0127:         "accuracy": float(accuracy_score(y[test], pred)),
0128:         "n_base": N_BASE,
0129:         "n_train": int(train.sum()),
0130:         "n_test": int(test.sum()),
0131:     })
0132: return rows
0133:
0134:
0135: def main():
0136:     OUT_DIR.mkdir(parents=True, exist_ok=True)
0137:     reset_full_dataset_seed()
0138:
0139:     pair_df = full.build_dataset()
0140:     prompts, pair_df = full.index_prompts(pair_df)
0141:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0142:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0143:
0144:     rows = []
0145:     for model_name in MODELS:
0146:         print(f"\n=== MODEL: {model_name} ===", flush=True)
0147:         pooled_vectors = get_pooled_vectors(model_name, prompts)
0148:
0149:         for pooling, vectors in pooled_vectors.items():
0150:             for representation in ["x_only", "y_only", "concat", "delta"]:
0151:                 rows.extend(run_one(model_name, pooling, representation, vectors, pair_df))
0152:
0153:     result = pd.DataFrame(rows)
0154:     result.to_csv(OUT_DIR / "full_semantic_pooling_ablation.csv", index=False)
0155:     pivot = result.pivot_table(
0156:         index=["model", "pooling", "classifier"],
0157:         columns="representation",
0158:         values="accuracy",
0159:     ).reset_index()
0160:     pivot.to_csv(OUT_DIR / "full_semantic_pooling_ablation_pivot.csv", index=False)
0161:     print(pivot)
0162:
```

**Code listing: run\_full\_semantic\_pooling\_ablation.py (4)**

```
0163:
0164: if __name__ == "__main__":
0165:     main()
```

## Code listing: analyze\_confusion\_negation.py (1)

```
0001: from pathlib import Path
0002:
0003: import pandas as pd
0004:
0005:
0006: ROOT = Path(__file__).resolve().parents[1]
0007: OUT = ROOT / "results"
0008: OUT.mkdir(exist_ok=True)
0009:
0010:
0011: def parse_name(path):
0012:     stem = path.stem.replace("confusion_", "")
0013:     if stem.endswith("_linear_svc"):
0014:         return stem[:-11].replace("__", "/"), "linear_svc"
0015:     if stem.endswith("_logreg"):
0016:         return stem[:-7].replace("__", "/"), "logreg"
0017:     raise ValueError(path.name)
0018:
0019:
0020: def load_confusion(path):
0021:     model, classifier = parse_name(path)
0022:     cm = pd.read_csv(path, index_col=0)
0023:     labels = list(cm.index)
0024:
0025:     rows = []
0026:     errors = []
0027:
0028:     for label in labels:
0029:         total = float(cm.loc[label].sum())
0030:         correct = float(cm.loc[label, label])
0031:         recall = correct / total if total else 0.0
0032:         rows.append({
0033:             "source": "full_semantic",
0034:             "model": model,
0035:             "classifier": classifier,
0036:             "class": label,
0037:             "support": int(total),
0038:             "correct": int(correct),
0039:             "recall": recall,
0040:             "error_rate": 1.0 - recall,
0041:             "is_negation": label == "negation",
0042:         })
0043:
0044:     for pred in labels:
0045:         if pred == label:
0046:             continue
0047:         count = int(cm.loc[label, pred])
0048:         if count:
0049:             errors.append({
0050:                 "source": "full_semantic",
0051:                 "model": model,
0052:                 "classifier": classifier,
0053:                 "true_class": label,
0054:                 "predicted_class": pred,
```

## Code listing: analyze\_confusion\_negation.py (2)

```
0055:         "count": count,
0056:         "true_support": int(total),
0057:         "share_of_true": count / total if total else 0.0,
0058:         "involves_negation": label == "negation" or pred == "negation",
0059:     })
0060:
0061:     return rows, errors
0062:
0063:
0064: def main():
0065:     paths = sorted((ROOT / "lie_llm_full_semantic_holdout_results").glob("confusion_*.csv"))
0066:     all_rows = []
0067:     all_errors = []
0068:
0069:     for path in paths:
0070:         rows, errors = load_confusion(path)
0071:         all_rows.extend(rows)
0072:         all_errors.extend(errors)
0073:
0074:     class_df = pd.DataFrame(all_rows)
0075:     error_df = pd.DataFrame(all_errors)
0076:
0077:     class_df.to_csv(OUT / "confusion_class_recall.csv", index=False)
0078:     error_df.sort_values(["count", "share_of_true"], ascending=False).to_csv(
0079:         OUT / "confusion_top_errors.csv",
0080:         index=False,
0081:     )
0082:
0083:     neg_summary = (
0084:         class_df
0085:         .assign(group=lambda d: d["is_negation"].map({True: "negation", False: "non_negation"}))
0086:         .groupby(["source", "model", "classifier", "group"])
0087:         .agg(
0088:             mean_recall=("recall", "mean"),
0089:             mean_error_rate=("error_rate", "mean"),
0090:             n_classes=("class", "count"),
0091:         )
0092:         .reset_index()
0093:     )
0094:     neg_summary.to_csv(OUT / "confusion_negation_summary.csv", index=False)
0095:
0096:     rank_df = class_df.copy()
0097:     rank_df["recall_rank_low_is_hard"] = rank_df.groupby(["model", "classifier"])["recall"].rank(method="min")
0098:     rank_df.sort_values(["model", "classifier", "recall"]).to_csv(
0099:         OUT / "confusion_class_recall_ranked.csv",
0100:         index=False,
0101:     )
0102:
0103:     print("Saved:")
0104:     for name in [
0105:         "confusion_class_recall.csv",
0106:         "confusion_top_errors.csv",
0107:         "confusion_negation_summary.csv",
0108:         "confusion_class_recall_ranked.csv",
```

## Code listing: analyze\_confusion\_negation.py (3)

```
0109:     ]:
0110:         print(OUT / name)
0111:
0112:
0113: if __name__ == "__main__":
0114:     main()
```

## Code listing: build\_signed\_permutation\_multiple\_testing.py (1)

```
0001: from pathlib import Path
0002:
0003: import numpy as np
0004: import pandas as pd
0005:
0006:
0007: ROOT = Path(__file__).resolve().parents[1]
0008: OUT = ROOT / "lie_algebraic_identities_results" / "csv"
0009:
0010:
0011: def benjamini_hochberg(p_values):
0012:     p = np.asarray(p_values, dtype=float)
0013:     n = len(p)
0014:     order = np.argsort(p)
0015:     ranked = p[order]
0016:     adj = np.empty(n, dtype=float)
0017:     prev = 1.0
0018:     for i in range(n - 1, -1, -1):
0019:         rank = i + 1
0020:         val = min(prev, ranked[i] * n / rank)
0021:         prev = val
0022:         adj[order[i]] = val
0023:     return np.clip(adj, 0, 1)
0024:
0025:
0026: def bootstrap_p_less_than_one(values, n_boot=20000, seed=42):
0027:     rng = np.random.default_rng(seed)
0028:     values = np.asarray(values, dtype=float)
0029:     idx = rng.integers(0, len(values), size=(n_boot, len(values)))
0030:     means = values[idx].mean(axis=1)
0031:     p = (np.sum(means >= 1.0) + 1) / (n_boot + 1)
0032:     return float(p)
0033:
0034:
0035: def main():
0036:     files = [
0037:         ROOT / "lie_algebraic_identities_results" / "csv" / "jacobi_raw_all_models.csv",
0038:         ROOT / "lie_algebraic_identities_decoder_results" / "csv" / "jacobi_raw_all_models.csv",
0039:     ]
0040:     raw = pd.concat([pd.read_csv(path) for path in files if path.exists()], ignore_index=True)
0041:
0042:     rows = []
0043:     for (model, triple), group in raw.groupby(["model", "triple"]):
0044:         ratios = group["jacobi_to_null_mean_ratio"].to_numpy()
0045:         mean_ratio = float(ratios.mean())
0046:         p_less = bootstrap_p_less_than_one(ratios)
0047:         rows.append({
0048:             "model": model,
0049:             "triple": triple,
0050:             "n": int(len(group)),
0051:             "mean_ratio_to_null": mean_ratio,
0052:             "bootstrap_p_ratio_less_than_1": p_less,
0053:             "direction": "below_null" if mean_ratio < 1 else "above_null",
0054:         })
```

## Code listing: build\_signed\_permutation\_multiple\_testing.py (2)

```
0055:
0056:     result = pd.DataFrame(rows).sort_values(["bootstrap_p_ratio_less_than_1", "model", "triple"])
0057:     result["bonferroni_p"] = np.minimum(result["bootstrap_p_ratio_less_than_1"] * len(result), 1.0)
0058:     result["bh_fdr_p"] = benjamini_hochberg(result["bootstrap_p_ratio_less_than_1"].to_numpy())
0059:     result["passes_bonferroni_0_05"] = result["bonferroni_p"] < 0.05
0060:     result["passes_bh_fdr_0_05"] = result["bh_fdr_p"] < 0.05
0061:
0062:     out_path = OUT / "signed_permutation_multiple_testing.csv"
0063:     result.to_csv(out_path, index=False)
0064:     print(out_path)
0065:     print(result)
0066:
0067:
0068: if __name__ == "__main__":
0069:     main()
```